LLM-Driven Adaptive Authentication and Access Control in Zero Trust Systems Using Blockchain

A PROJECT REPORT

Submitted by:

23BIS70035 Parth Kansal

23BIS70073 Anshul Sharma

23BIS70081 Vireshwar Singh Tomar

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE WITH SPECIALIZATION IN INFORMATION SECURITY

Under the Supervision of:

Sidrah Fayaz Wani (E17441)



Chandigarh University, Gharuan, Mohali - 140413, Punjab

January 2026

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

BONAFIDE CERTIFICATE

Certified that this project report **LLM-Driven Adaptive Authentication and Access Control in Zero Trust Systems Using Blockchain** is the Bonafide work of **Parth Kansal (23BIS70035), Anshul Sharma (23BIS70073), Vireshwar Singh Tomar(23BIS70081)** who carried out the project work under my/our supervision.

SIGNATURE	**SIGNATURE**
Dr. Aman Kaushik (E5942)	**Sidrah Fayaz Wani (E17441)**
	SUPERVISOR
HEAD OF THE DEPARTMENT	Assistant Professor
AIT-CSE(Core)	**AIT-CSE(Core)**

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER EXTERNAL EXAMINER

Table of Contents

ABSTRACT 1
GRAPHICAL ABSTRACT 2
LIST OF FIGURES 3
LIST OF TABLES 4
CHAPTER 1 5
INTRODUCTION 5
1.1 Problem Definition 5
1.2 Problem Overview 6
1.3 Task Identification 6
1.4 Timeline 7
CHAPTER 2 8
LITERATURE SURVEY 8
2.1 Literature survey 8
2.2 Research Gaps 9
2.3 Top Papers to Read: 9
2.4 Research Paper Summaries 10
2.5 Datasets and Tools 12
CHAPTER 3 14
DESIGN FLOW/PROCESS 14
3.1 Concept Generation 14
3.2 Proposed Concept 14
3.2.1 Machine Learning-Based Risk Scoring 14
3.2.2 Blockchain-Based Audit and Trust Mechanism 18
3.2.3 AI-Assisted Voting-Based Authorization System 21
3.3 Design Constraints 23
3.4 Alternative Designs 25
3.4.1 Design 1: Fully Automated AI-Based System 25

3.4.2 Design 2: Hybrid AI + Voting-Based System (Proposed) 26
3.6 System Architecture Overview.....31
3.7 Design Flow (Algorithm) 34
3.8 Flowcharts 37
3.8.1 Sequence Diagram (Step-by-Step Execution) 37
3.8.2 DFD Level 0 (Context Diagram).....37
3.8.3 DFD Level 1 (Detailed System Flow).....38
3.8.4 Use Case Diagram.....38
3.8.4 Use Case Diagram.....39
3.9 Implementation Plan.....40
CHAPTER 4.....55
RESULTS ANALYSIS AND VALIDATION.....55
4.1 Implementation Using Modern Engineering Tools.....55
4.2 System Design and Architecture 56
4.2.1 Overall System Workflow 57
4.2.2 Key Architectural Components 57
4.2.3 Functional Description of Modules 57
4.3 Testing and Validation 59
4.4 Results and Observations 60
4.5 Data Analysis and Interpretation.....60
4.6 Validation of the System 61
4.7 Challenges and Limitations.....62
CHAPTER 5.....63
5.1 Conclusion.....63
5.2 Deviation from Expected Results.....64
5.3 Future Work (Way Ahead).....66
5.4 Final Remarks.....68
5.5 References 68

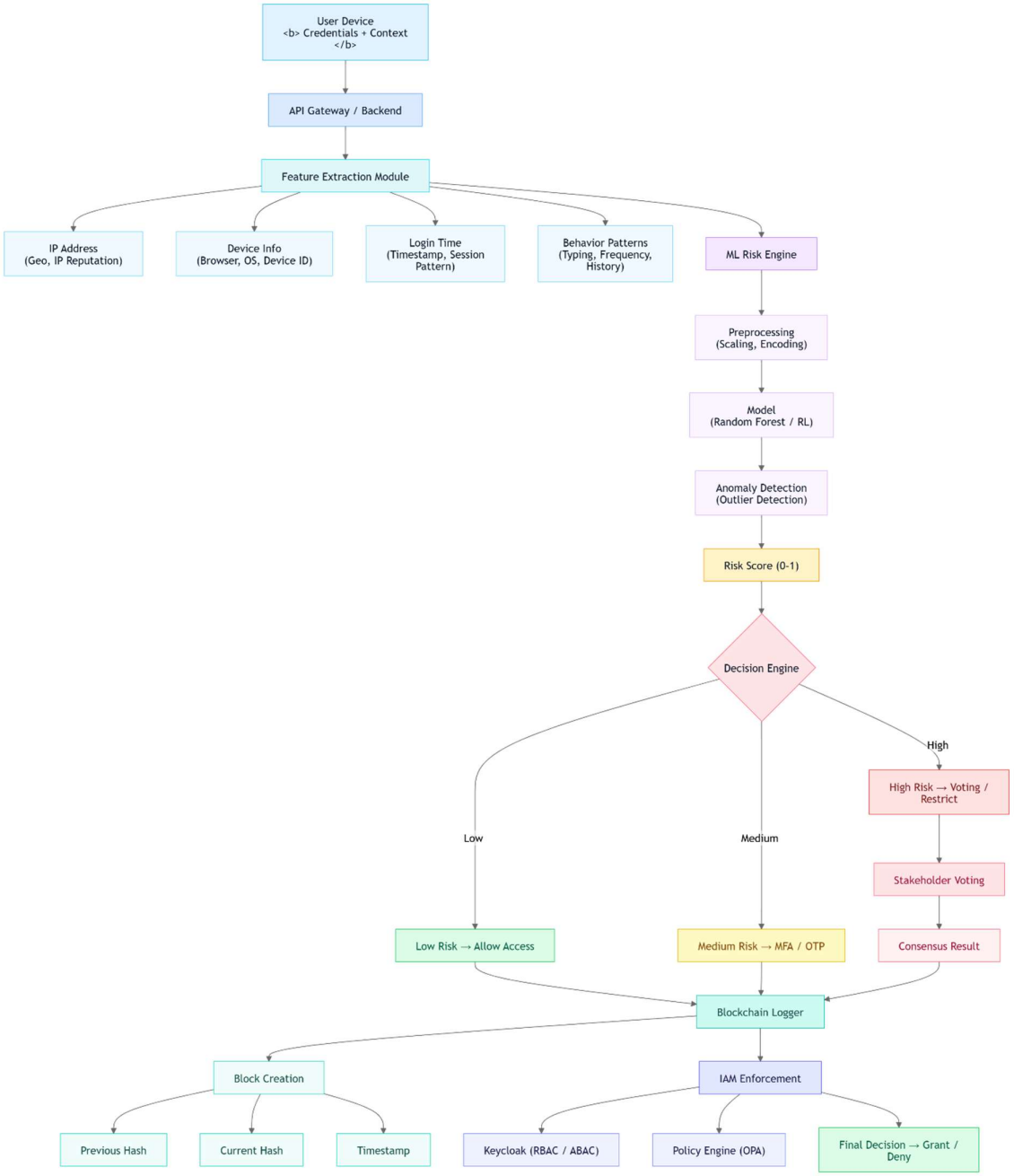
ABSTRACT

Traditional Identity and Access Management (IAM) systems rely heavily on static authentication and authorization mechanisms, which are increasingly ineffective against modern cybersecurity threats such as credential theft, insider attacks, and unauthorized data access. These systems often apply uniform security controls regardless of contextual risk, leading either to weakened security or unnecessary user friction. Moreover, centralized identity storage introduces single points of failure, reducing transparency and trust.

This project proposes an intelligent, adaptive, and decentralized IAM framework aligned with Zero Trust principles by integrating dynamic risk-based authentication, Large Language Models (LLMs), and blockchain technology. The system continuously evaluates contextual and behavioural parameters—including user behaviour, device characteristics, location, access time, and resource sensitivity—to compute real-time risk scores. Based on the assessed risk level, authentication strength is dynamically adjusted using adaptive multi-factor authentication (MFA). LLMs are employed to provide contextual reasoning for access decisions and to assist in intelligent approval workflows for sensitive or unauthorized access requests. Blockchain technology ensures tamper-resistant storage of identity proofs, access approvals, and audit logs, thereby enhancing transparency, integrity, and non-repudiation.

Experimental evaluation demonstrates that the proposed framework significantly improves access control adaptability, reduces administrative overhead, and strengthens security while maintaining a smooth user experience. The results indicate that combining AI-driven decision support with decentralized identity management offers a scalable and effective solution for next-generation IAM systems in modern digital environments.

GRAPHICAL ABSTRACT



This diagram illustrates an AI-driven risk-based authentication system. User data is analysed through feature extraction and machine learning to generate a risk score. Based on risk levels, access is allowed, challenged via MFA, or restricted with voting. All actions are securely logged using blockchain and enforced through an IAM system.

LIST OF FIGURES

- Figure 1: AI-driven risk-based authentication system
- Figure 3.4.1: Decision constraint
- Figure 3.4.2: Ai based voting system
- Figure 3.6.1: Risk Factors Evaluation
- Figure 3.9.1: Authentication Process
- Figure 3.9.2: Risk evaluation feature extraction
- Figure 3.9.3: Storing Events in a Blockchain
- Figure 3.9.5: Advance voting mechanism
- Figure 4.1: Login Process

LIST OF TABLES

Table 1: Summary of Research Papers on Risk-Based Authentication and Access Control

Table 2: Datasets, Tools, and Platforms Used

Table 3: User Context Parameters

Table 4: Behavioural Parameters

Table 5: Device & Network Trust Parameters

Table 6: Historical Risk Parameters

Table 7: Access Request Parameters

Table 8: Derived / Engineered Features

Table 9: Risk-Based Access Control Decision Mapping

Table 10: Blockchain Log Data Schema for Authentication and Access Events

Table 11: Technologies Used

Table 12: Key Architectural Components

Table 13: Performance Metrics

Table 14: Validation Criteria

CHAPTER 1
INTRODUCTION

1.1 Problem Definition

In today's digital world, organizations heavily depend on technology, especially the Internet, to manage users, data, and resources. With the rapid increase in usage, the volume of data and services has grown significantly. Every modern system includes authentication and authorization mechanisms to ensure security and proper access control.

Authentication is the process of verifying a user's identity before granting access to resources. It ensures that the user is genuinely who they claim to be, using credentials such as usernames and passwords, biometrics, or one-time passwords (OTP). Authorization determines what actions a user is allowed to perform after authentication. It defines the permissions and access levels assigned to the user. Authorization always follows authentication.

However, most current authentication and authorization systems rely on predefined static rules. While these approaches work in simple environments, they struggle to adapt to changing user behaviour, varying risk levels, and modern security threats such as stolen credentials, insider threats, and unauthorized data access.

Current Identity and Access Management (IAM) systems often treat all users in the same way, apply identical authentication requirements regardless of the actual risk involved. This can either weaken security (if controls are too relaxed) or create unnecessary friction for users (if controls are too strict). Additionally, the process of granting or revoking access is often manual, time-consuming, and lacks transparency and accountability. Centralized storage of user data further introduces risks of creating a single point of failure, which can be exploited through data breaches or manipulation.

Static authorization models are also limited, as they cannot efficiently handle dynamic environments where user roles, contexts, and conditions frequently change. As a result, there is a clear need for an intelligent, adaptive, and secure access control system. Such a system should be capable of:

1. Evaluating user risk in real time
2. Dynamically adjusting authentication strength
3. Providing transparent and tamper-resistant identity management
4. Maintaining a smooth and trustworthy user experience

In conclusion, authentication and authorization remain essential components of secure systems, but **modern challenges demand adaptive, risk-aware, and scalable solutions to ensure both security and usability.**

1.2 Problem Overview

As increasingly rely on digital systems, managing who can access data and which type of data and resources has become more critical than ever. Most existing access control systems are built on predefined rules that does not change according to user behaviour or risk. Thus, make ineffective against modern security threats such as stolen credentials, insider threats or unauthorized access.

Current IAM solutions apply the same security checks to every user, regardless of the situation. While this can make systems safer, it also frustrates legitimate users with unnecessary authentication steps. At the same time, approving access to sensitive information is often a slow and manual process, lacking transparency and clear accountability. Also, user identities are stored in centralized systems which increases the risk of large-scale data breaches.

Because of these limitations, there is a growing need for a smarter and more flexible access control approach. Thus, there should be an effective solution to understand context, assess risk dynamically, and apply appropriate security measures while ensuring transparency, trust, and a smooth user experience.

1.3 Task Identification

Based on the limitations of traditional authentication and authorization systems, the primary task is to design and develop an intelligent, adaptive, and secure system that overcomes the drawbacks of static rule-based approaches.

The system should be capable of analysing user behaviour, context, and risk levels in real time to make dynamic access decisions. Unlike conventional systems, it must not treat all users equally but instead adjust authentication mechanisms and access permissions based on the level of risk involved.

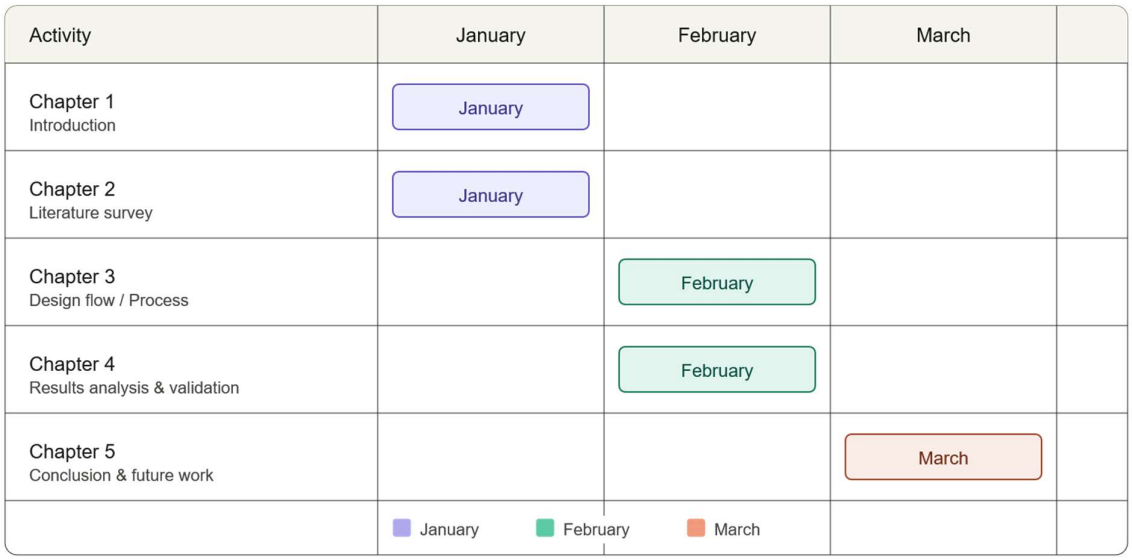
The key tasks involved in this system include:

- Developing a dynamic authentication mechanism that can adapt (e.g., require OTP, biometric, or multi-factor authentication based on risk level).
- Implementing risk-based authorization to control user access depending on behaviour, location, device, or activity patterns.

- Ease access provisioning and revocation, reducing manual effort and improving efficiency.
- Ensuring transparency and accountability by maintaining secure logs and audit trails of all access-related activities.
- Reducing dependency on centralized systems by exploring secure and tamper-resistant approaches for identity management.
- Enhancing overall security while maintaining a smooth and user-friendly experience.

The goal of this task is to build a **Next-generation Identity and Access Management (N-IAM)** system that is secure, scalable, adaptive, and capable of handling modern cybersecurity challenges effectively.

1.4 Timeline



CHAPTER 2
LITERATURE SURVEY

2.1 Literature survey

Adaptive, AI-driven authentication is a **growing focus** for securing sensitive data. Recent research emphasizes **risk-based authentication (RBA)**, which dynamically adjusts login challenges by computing a risk score from user behavior and context[1][2]. For example, Wiefling *et al.* (2022) present a large-scale RBA study (3.3M users, 31M logins) and offer an **open synthesized login dataset**[3][4]. Picard and Pierre (2023) propose **RLAuth**, a deep-reinforcement-learning RBA system for mobile, achieving a 92.6% G-Mean on public smartphone data[2][5]. Fereidouni *et al.* (2024) introduce **F-RBA**, a federated-learning RBA framework that trains risk models locally on devices (preserving privacy) and outperforms conventional anomaly detectors in true-positive rate[6]. These efforts highlight a **shift toward ML/FL-based, contextual, continuous authentication**.

Behavioral and continuous authentication (e.g. via sensors or biometrics) is surveyed by Baig & Eskeland (2021), noting privacy–security trade-offs of passive user monitoring[7]. Such methods can complement RBA by continuously verifying identity during a session.

For **trust/risk scoring**, Wang *et al.* (2023) propose a zero-trust model (AU-ZTAC) that computes user trust scores for fine-grained IoV (vehicle) access control[8]. This demonstrates the use of attribute-based trust metrics to reflect user intent in access decisions.

On the **blockchain** side, several frameworks demonstrate immutable logging and decentralized access control. Hu *et al.* (2023) present "N-Accesses", using Ethereum smart contracts to enforce limited (n-times) data access and audit logs[9]. Yaqub *et al.* (2024) describe a **policy-based AC for healthcare**, where smart contracts enforce consent and GDPR-compliant data sharing[10]. Tawfik *et al.* (2025) propose **ACHealthChain** (on Hyperledger Fabric and IPFS) with fine-grained "PolicyChain" and "LogChain" modules, greatly improving throughput and auditability over prior systems[11]. These show blockchain's strengths in tamper-proof logging, distributed policy enforcement, and privacy-preserving data sharing.

Finally, **human-in-the-loop governance** is an emerging idea: e.g. flagging anomalous requests to a panel of stakeholders. No standard architecture exists yet, but analogous concepts appear in AI/ML governance literature (e.g. "oversight" to ensure trust), suggesting policy committees or DAO-style voting could be applied to sensitive access requests.

2.2 Research Gaps

We found few works combining RBA with blockchain or with human review. Specifically, there is little *explanation for AI for access control* or *integrated human voting* in an ML + blockchain authentication system. This gap suggests opportunity to design a transparent, auditable pipeline: ML risk-scoring + dynamic auth → blockchain log → (if needed) human committee vetting.

2.3 Top Papers to Read:

1. **Wiefling *et al.* 2022 (ACM TAPSec)** – Empirical RBA study; provides synthetic login dataset[3][4].
2. **Picard & Pierre 2023 (IEEE Access)** – RLAuth: Reinforcement-learning RBA with strong results[2][5].
3. **Fereidouni *et al.* 2024 (arXiv)** – F-RBA: Federated RBA framework improving privacy[6].
4. **Hu *et al.* 2023 (Sensors)** – "N-Accesses": Blockchain-based IoT access control with smart contracts and audits[9].
5. **Tawfik *et al.* 2025 (Sci. Reports)** – ACHealthChain: Hyperledger/IPFS AC for healthcare, fine-grained policies, scalable performance[11].
6. **Yaqub *et al.* 2024 (PeerJ CS)** – PBAC with blockchain for EHRs, smart-contract enforcement, GDPR compliance[10].
7. **Wang *et al.* 2023 (Electronics)** – AU-ZTAC: Trust-score zero-trust model combining attributes and dynamic trust for IoV[8].
8. **Zisad 2026 (arXiv)** – LLMAC: LLM-based unified AC (RBAC/ABAC/DAC) with explanations (98.5% acc)[12].

These cover core components (RBA/ML, blockchain AC, trust scoring, explainability). Our project design will build on their methods and data.

Table 1: Summary of Research Papers on Risk-Based Authentication and Access Control

Topic	Citation	Year	Relevance
Risk-Based Auth (RBA) Survey	Wiefling *et al.* – *Pump Up Password Security!* (ACM TAPSec)[3]	2022	5
RBA via Reinforcement Learning	Picard & Pierre – *RLAuth* (IEEE Access)[2]	2023	5
Federated RBA (F-RBA)	Fereidouni *et al.* – *F-RBA* (arXiv)[6]	2024	4
Behavioral/Continuous Auth Survey	Baig & Eskeland – *Cont. Auth Survey* (Sensors)[7]	2021	4
User Trust-Score (Zero-Trust AC)	Wang *et al.* – *AU-ZTAC* (Electronics)[8]	2023	3
Blockchain IoT AC ("N-Accesses")	Hu *et al.* – *Sensors* 2023[9]	2023	4
Blockchain Healthcare AC (PBAC)	Yaqub *et al.* – *PeerJ CS* 2025[10]	2025	4
Blockchain Healthcare AC (Fabric)	Tawfik *et al.* – *ACHealthChain* (Sci. Rep.)[11]	2025	4
Explainable AC with LLM	Zisad – *LLMAC* (arXiv)[12]	2026	3
Explainable AC (theory)	Mehri *et al.* – *Towards Explainable AC* (SACMAT)	2025	2
Anomaly Detection (general)	Picard & Pierre – *RLAuth* (IEEE Access)[2]	2023	4

2.4 Research Paper Summaries

- **Wiefling *et al.* (2022)** – *"Pump Up Password Security!"* (ACM Trans. on Privacy & Security)[3]. Large-scale empirical RBA study: 3.3M users, 31M logins over 1 year. Examines RBA configurations and ML optimization to balance security/usability, and even replaces IP with RTT for privacy. Key deliverables include an **open synthesized login dataset** and ML-tuned risk parameters[3][4]. Relevance: provides data & insights for building ML-driven risk scoring.
- **Picard & Pierre (2023)** – *"RLAuth: A Risk-Based Authentication System Using Reinforcement Learning"* (IEEE Access)[2][5]. Proposes RLAuth: a *mobile* RBA system using deep reinforcement learning (DRL) for anomaly detection. The DRL agent

dynamically selects authentication challenges (e.g. biometric vs PIN) based on context and data sensitivity. Introduces balanced sampling to handle class imbalance. **Results:** On the CRAWDAD *Telefonica MobilePhoneUse* dataset (42 users, ~0.9M samples), RLAuth achieves 92.6% G-mean, outperforming standard classifiers[2]. Relevance: shows ML (RL) can drive adaptive auth and continuous verification on smartphones.

- **Fereidouni *et al.* (2024)** – *"F-RBA: A Federated Learning-based Framework for Risk-based Authentication"* (arXiv)[6]. Introduces F-RBA, which trains RBA models via federated learning, keeping user data on-device for privacy. Novelty: a similarity-based feature engineering for heterogeneous data. **Results:** On a real-world multi-user login dataset, F-RBA yields higher true-positive rates for detecting suspicious logins than traditional unsupervised methods[6]. Relevance: federated approach informs privacy-preserving ML for risk scoring.
- **Baig & Eskeland (2021)** – *"Security, Privacy, and Usability in Continuous Authentication: A Survey"* (Sensors)[7]. Survey of continuous/passive auth methods (biometric, behavior, context). Discusses security challenges (e.g. replay attacks) and privacy issues (constant sensing). Relevance: background on continuous authentication, illustrating trade-offs when using sensor/behavior data for ongoing user verification.
- **Wang *et al.* (2023)** – *"AU-ZTAC: Attribute and User Trust Score-Based Zero Trust Access Control in IoV"* (Electronics)[8]. Proposes a Zero-Trust model that computes a **user trust score** from behavior and attributes, integrated with ABAC to decide access in vehicle networks. Experiments show the trust-based approach yields flexible, fine-grained control and enhances IoV security[8]. Relevance: examples of dynamic trust scoring (though IoV-focused) applicable to our trust-risk model.
- **Hu *et al.* (2023)** – *"N-Accesses"* (Sensors)[9]. Blockchain-enabled access control for IoT data. Data owners host files off-chain; smart contracts govern **n-times access tokens** with deadlines using hash-chains. Key benefit: smart contracts enforce token usage limits and immutably log all accesses. Performance evaluation (via simulation) shows feasible overhead. Relevance: demonstrates blockchain + smart contracts for fine-grained, auditable access control.
- **Yaqub *et al.* (2024)** – *"Blockchain-enabled Policy-Based Access Control for EHR"* (PeerJ CS)[10]. Access control for healthcare data: Subjects/controllers define policies, and smart contracts grant/revoke/check access per GDPR. Everyone (subject, controller, requester) can audit logs via blockchain. Shows blockchain empowers dynamic consent and

transparent privacy enforcement. Relevance: example of integrating policy-based AC and immutable logging via blockchain in a regulated domain.

- **Tawfik *et al.* (2025)** – *"ACHealthChain"* (Sci. Reports)[11]. Hyperledger Fabric + IPFS framework for healthcare data sharing. Introduces **PolicyChain** (fine-grained policy and revocation) and **LogChain** (audit trail). Patient data is sharded into EHR and diagnosis chains. Experiments: 19.7% throughput gain and 87% latency reduction vs. baseline. Relevance: a high-performance permissioned-blockchain AC system with multi-chain logging.
- **Zisad (2026)** – *"LLMAC: A Global and Explainable Access Control Framework with LLM"* (arXiv)[12]. Introduces LLMAC, an LLM-based model that unifies RBAC, ABAC, and DAC. Trained on synthetic policy scenarios, it achieves **98.5% accuracy** (vs 14–58% for individual models) and generates human-readable explanations for each decision[12]. Relevance: shows potential of AI (LLMs) to interpret and explain complex access policies.

2.5 Datasets and Tools

Table 2: Datasets, Tools, and Platforms Used

Name	Type	License	Relevance
RBA Login Dataset (Wiefling et al.)[4]	Dataset	CC BY 4.0	33M+ synthetic login attempts (IP, UA, etc.) for risk scoring.
MobilePhoneUse (Telefonica)[5]	Dataset	Non-commercial (CRAWDAD)	0.9M samples of real smartphone usage for continuous auth/AD.
IBM Bad Password Dataset (CIS)	Dataset	Corporate	Historic corpus of breached passwords for anomaly detection (if needed).
Simulated Anomaly Logs	(Simulated)	—	Could generate login/behavior data with tools like [68†L238-L247] for testing.
scikit-learn	ML Library	BSD	Classic ML algorithms (RF, XGBoost, one-class SVM) for risk modeling.

PyTorch / TensorFlow	ML Library	BSD/Apache 2.0	Deep learning frameworks for neural or RL-based auth modules.
Hyperledger Fabric	Blockchain Plat.	Apache 2.0	Permissioned blockchain; modular consensus, channels (used by ACHealthChain)[11].
Ethereum + Solidity	Blockchain Plat.	GPL/MIT	Public blockchain for smart contracts (e.g. "N-Accesses" scheme)[9].
Aragon (DAO)	Governance Tool	GPLv3	On-chain voting/governance framework (can prototype stakeholder voting).
Keycloak (IdentityServer)	IAM System	Apache 2.0	Open-source identity and access management platform (RBAC/ABAC support).
Open Policy Agent (OPA)	Policy Engine	Apache 2.0	Policy-as-code engine (Rego) to enforce access rules in microservices.
PuLP, Pandas	Python Tools	MIT	For dataset generation/analysis and ML pipeline management.

CHAPTER 3
DESIGN FLOW/PROCESS

3.1 Concept Generation

Currently, most systems use static authentication mechanisms, meaning that every user must go through the same login process regardless of their identity, behaviour, or context. Typically, this involves entering a username and password, and an additional verification step such as a one-time password (OTP). These methods treat all users equally, without considering whether the login attempt is coming from a trusted device, a familiar location, or a suspicious environment.

However, this uniform approach presents significant security limitations. Static authentication systems are unable to adapt to changing user behaviour or detect anomalies effectively. For example, if an attacker gains access to valid credentials, the system cannot distinguish between a legitimate user and a malicious actor. Similarly, insider threats, where authorized users misuse their access privileges, are difficult to detect because the system inherently trusts authenticated users without evaluating their intent or behaviour.

Moreover, static systems ignore important contextual factors such as login time, geographic location, device information, and historical usage patterns. A login attempt at an unusual hour or from an unknown device is treated the same as a routine login, which increases vulnerability to unauthorized access. As a result, these systems lack the flexibility and intelligence required to handle modern security challenges. Therefore, there is a growing need for dynamic and adaptive authentication mechanisms that can evaluate user risk in real time and adjust security measures accordingly. Such systems can significantly enhance security by incorporating behavioural analysis, contextual awareness, and intelligent decision-making into the authentication process.

3.2 Proposed Concept

3.2.1 Machine Learning-Based Risk Scoring

The proposed system replaces traditional rule-based authentication with a machine learning-driven approach to dynamically evaluate user risk. Instead of applying fixed authentication rules to every user, the system will analyse multiple contextual and behavioural factors to determine the legitimacy of each login attempt.

The machine learning model considers parameters as input. Type of parameters will be used are:

1. **User Context Parameters:** These describe the current login/request environment

Table 3: User Context Parameters

Parameter	Type	Description
login_time	datetime	Time of login
login_hour	integer (0–23)	Extracted hour
day_of_week	integer (0–6)	Pattern detection
ip_address	string	User IP
geo_location	categorical	Location derived from IP
device_type	categorical	Device category
browser	categorical	Browser info
os_type	categorical	Operating system

2. **Behavioural Parameters:** These tracks **how user behaves over time**

Table 4: Behavioural Parameters

Parameter	Type	Description
avg_login_time	float	Average login hour
login_time_deviation	float	Difference from normal
login_frequency	integer	Number of logins
failed_attempts	integer	Failed login attempts
session_duration	float	Time spent per session
typing_speed	float	Optional behavioral biometrics

3. **Device & Network Trust Parameters:** Used to check familiarity and trust level

Table 5: Device & Network Trust Parameters

Parameter	Type	Description
known_device	boolean	Previously used device
known_ip	boolean	Known IP or not
vpn_detected	boolean	VPN usage
proxy_detected	boolean	Proxy detection
device_change_count	integer	Device switching frequency

4. **Historical Risk Parameters:** Past system records

Table 6: Historical Risk Parameters

Parameter	Type	Description
past_risk_score	float (0–1)	Previous risk
suspicious_activity_count	integer	Past alerts
last_login_gap	float (hours)	Time since last login
account_age	integer (days)	Age of account
role	categorical	Access level

5. **Access Request Parameters (for file access):** Used when user requests data

Table 7: Access Request Parameters

Parameter	Type	Description
resource_requested	string	File name
sensitivity_level	categorical	Data importance
request_time	datetime	Time of request
reason_text	string (NLP input)	User justification
reason_validity_score	float (0–1)	Output from NLP model

6. **Derived / Engineered Features:** These are calculated features (important for ML)

Table 8: Derived / Engineered Features

Parameter	Type	Description
anomaly_score	float (0–1)	Overall deviation
trust_score	float (0–1)	User trust level
risk_score	float (0–1)	Final output
severity_level	categorical	Final classification

These all parameters are taken as input. Let the input feature vector be represented as:

$$X = \{x_1, x_2, x_3, \dots, x_n\}$$

where each x_i corresponds to a feature such as login time, device type, IP address, location, past behavior, or failed attempts.

Risk Score Computation

The machine learning model maps the input features to a probability score:

$$S = f(X), S \in [0,1]$$

Where S= Risk score, f= learned model (e.g., Logistic Regression, Random Forest, Neural Network).

This score represents the likelihood that a login attempt is anomalous or risky.

Probabilistic Interpretation

In many implementations, especially with logistic regression, the score can be interpreted as:

$$S = P(X)$$

Meaning:

- $S \approx 0 \rightarrow$ Highly legitimate
- $S \approx 1 \rightarrow$ Highly suspicious

Risk Categorization

The continuous score is converted into discrete categories:

$$Category = \{Low\ S < 0.3 \quad Medium\ 0.3 \leq S < 0.7 \quad High\ S \geq 0.7$$

Adaptive Authentication Mechanism

Based on the computed risk score, the system dynamically adjusts authentication requirements:

Table 9: Risk-Based Access Control Decision Mapping

Risk Level	System Action
Low	Direct access
Medium	Additional verification (OTP / MFA)
High	Restrict / trigger voting

This can be represented as a decision function:

$$A = g(S)$$

where A is the authentication action.

Learning and Adaptation

The model continuously improves by learning from historical data. Let:

$$D = \{(X_1, y_1), (X_2, y_2), \dots, (X_m, y_m)\}$$

Where X_i= input features, $y_i \in \{0,1\}$= label (legitimate or suspicious)

The model updates its parameters θ by minimizing a loss function:

$$\theta^* = arg \ _\theta \sum_{i=1}^{m} \ L(f_\theta(X_i), y_i)$$

Anomaly Detection Capability

The system detects unusual behaviour by measuring deviation:

$$Anomaly\ Score = | X - \mu |$$

where μ represents normal user behavior.

Larger deviation → higher risk

3.2.2 Blockchain-Based Audit and Trust Mechanism

Need for Blockchain in the Proposed System

In traditional systems, logs and audit records are typically stored in centralized databases such as SQL or NoSQL systems. While these systems are efficient, they suffer from critical security limitations such as unauthorized modification of logs, deletion of audit records, and lack of traceability. The data stored in such databases can be modified, deleted, or tampered by administrators or attackers who gain sufficient privileges. This creates a major issue in security-sensitive environments where **trust, accountability, and traceability** are essential.

For example, if a malicious user gains administrative access, they can alter login records or delete traces of unauthorized access. This makes it difficult to detect breaches and perform forensic analysis.

To overcome it, the proposed system integrates a **blockchain-based audit mechanism**, which ensures that all critical system events are recorded in a secure and tamper-proof manner.

Blockchain is a distributed ledger technology where data is stored in the form of **blocks**, and each block is linked to the previous one using cryptographic hashes. This creates a chain-like structure where altering any previous record would require changing all subsequent blocks, which is computationally infeasible.

In this system, blockchain is used **not for cryptocurrency**, but for secure logging, event tracking, trust establishment

Key Properties Utilized

1. **Immutability:** Once a record is written to the blockchain, it cannot be modified or deleted. This means login records are permanent, access decisions cannot be altered, and full audit history is preserved
2. **Transparency:** All authorized participants can view the recorded transactions. Means admins can verify actions, Stakeholders can audit decisions and there will no hidden modifications
3. **Decentralization:** Data is not stored in a single location but distributed across nodes. Thus, there is no single point of failure, and no single authority can manipulate data.
4. **Cryptographic Security:** Each block is secured using hashing algorithms. This guarantees data integrity and any tampering is instantly detectable.

What Data on Blockchain: The system logs **only critical events**, not entire datasets (to optimize performance and cost).

Stored Data Fields:

Table 10: Blockchain Log Data Schema for Authentication and Access Events

Field	Type	Description
user_id	string	Unique user identifier
action_type	categorical	Login / Access Request / Vote
resource_id	string	Requested file/resource
risk_score	float	ML-generated risk
decision	categorical	Approved / Denied
timestamp	datetime	Event time
previous_hash	string	Link to previous block
current_hash	string	Hash of current block

Working Process

The blockchain-based audit mechanism in the proposed system follows a structured, step-by-step process to securely record and preserve all critical system events.

1. The process begins when an event occurs within the system, such as a user login attempt or a file access request. These events act as triggers for initiating the blockchain logging procedure. These events can be mathematically represented as:

$$E = \{U, A, R, t\}$$

where U denotes the user identity, A represents the action (login, request, or vote), R refers to the resource involved, and t is the timestamp of the event.

2. Once an event is detected, the system proceeds to the data collection phase. In this step, all relevant information associated with the event is gathered, including user identification details, the type of action performed (e.g., login or file request), and the risk score generated by the machine learning model. This ensures that each event is recorded with sufficient context for future analysis and auditing. This can be expressed as:

$$D = f(E, S)$$

where S is the risk score and D is the structured data prepared for storage.

3. After collecting the required data, the system creates a new block. This block contains the event data, a timestamp indicating when the event occurred, and the hash of the previous block in the chain. The inclusion of the previous hash ensures continuity and linkage between blocks, forming a secure chain structure. Each block in the blockchain is defined as:

$$B_i = \{D_i, T_i, H_{i-1}\}$$

where D_i is the data of the current block, T_i is the timestamp, and H_{i-1} is the hash of the previous block. This ensures that each block is linked to its predecessor.

4. Next, a cryptographic hash is generated for the newly created block using a hashing algorithm such as SHA-256:

$$H_i = SHA256(D_i \parallel T_i \parallel H_{i-1})$$

This hash acts as a unique digital fingerprint of the block's contents. Even a minor change in the data would result in a completely different hash value, thereby ensuring data integrity and making tampering easily detectable.

5. Once the hash is generated, the block is added to the blockchain. It is securely linked to the previous block using the stored hash, thereby extending the chain. This linkage ensures that any attempt to alter a previous block would break the chain, making such tampering practically infeasible.
6. In distributed implementations, the updated blockchain is then replicated across multiple nodes in the network. This replication ensures decentralization, meaning that no single entity has full control over the data. It also enhances reliability and fault tolerance, as the data is available across multiple locations.

Through this systematic process, the blockchain ensures that all system activities are recorded in a secure, transparent, and tamper-proof manner, thereby strengthening trust, accountability, and auditability within the system.

3.2.3 AI-Assisted Voting-Based Authorization System

The proposed system introduces an AI-assisted voting-based authorization mechanism to handle sensitive access requests in a secure and intelligent manner. This approach combines the computational efficiency of artificial intelligence with the contextual judgment of human stakeholders, ensuring both speed and reliability in decision-making. When a user submits a request to access a resource, the system first evaluates the request using an AI model that considers two primary parameters: the user's risk score S and the validity of the provided reason R. These inputs can be represented as a decision function:

$$D_{AI} = f(S, R)$$

where D_{AI}represents the preliminary decision generated by the AI model.

If the risk score is below a predefined threshold $\alpha$and the reason validity score exceeds a threshold β, the system directly grants access without additional intervention. This condition can be expressed as:

$$S < \alpha \ \wedge \ R > \beta \Rightarrow D = Approve$$

This ensures that legitimate users with valid intentions experience minimal friction, thereby maintaining system efficiency.

However, in scenarios where the request is considered uncertain or potentially risky—such as when $S \geq \alpha$ or $R \leq \beta$ —the system activates a collaborative voting mechanism:

$$S \geq \alpha \ \vee \ R \leq \beta \Rightarrow Voting\ Activated$$

In this phase, a set of authorized participants is selected:

$$V = \{v_1, v_2, \dots, v_n\}$$

where each participant v_i represents a stakeholder such as the data owner, administrator, or trusted user. Each voter evaluates the request and casts a binary decision:

$$v_i \in \{0,1\}$$

where 1 denotes approval and 0 denotes denial.

The final decision is computed using a majority voting rule, ensuring fairness and collective agreement:

$$D_{final} = \begin{cases} Approve, \sum_{i=1}^{n} v_i > \frac{n}{2} \\ Deny, otherwise \end{cases}$$

This mechanism ensures that no single entity has unilateral control over sensitive decisions, thereby increasing accountability and reducing the risk of misuse.
Additionally, the confidence of the voting process can be quantified as:

$$C = \frac{\sum_{i=1}^{n} v_i}{n}$$

where C represents the proportion of approving votes.

A higher value of Cindicates stronger consensus among stakeholders. Once the final decision D_{final} is determined, the entire process—including user details, risk score, reason score, voting results, and timestamp—is securely recorded on the blockchain:

$$B_i = h(U, S, R, D_{final}, t)$$

where $h(\cdot)$is a cryptographic hash function.

This ensures that the decision is immutable, transparent, and auditable. This hybrid system effectively addresses the limitations of standalone AI or purely human-based decision-making. While AI provides rapid and scalable evaluation through functions like $f(S, R)$, it may lack contextual understanding in ambiguous cases. Conversely, human stakeholders offer better judgment but are slower and less scalable. By combining both approaches, the system achieves an optimal balance between automation and human oversight.

Thus, the AI-assisted voting mechanism introduces a robust, adaptive, and trustworthy access control model that minimizes unauthorized access while maintaining operational efficiency and transparency.

3.3 Design Constraints

The proposed AI-driven authentication and authorization system operates under several critical design constraints that influence its technical architecture, scalability, and real-world applicability. These constraints are not only technical but also legal, economic, and societal in nature, requiring a balanced and well-informed design approach.

1. **Regulatory and Privacy Constraints:**

   One of the most significant constraints is compliance with data protection regulations such as the General Data Protection Regulation (GDPR) and similar frameworks. Since the system processes sensitive user data—including identity credentials, behavioral patterns, and access logs—it must ensure strict privacy protection. This introduces limitations on how data can be collected, stored, and analyzed. For example, continuous behavioral monitoring improves security but raises privacy concerns, requiring mechanisms such as user consent, anonymization, and encryption. Additionally, features like the right to access and delete data must be incorporated, which impacts database design and storage strategies.

2. **Economic and Resource Constraints:**

   The integration of blockchain and machine learning introduces both computational and financial overhead. Blockchain systems, particularly public ones, can incur transaction costs and scalability limitations, while private blockchains require dedicated infrastructure and maintenance. Similarly, machine learning models require substantial computational resources for training and periodic updates. This creates a trade-off between model complexity and system efficiency. A highly accurate but resource-intensive model may not be practical for real-time authentication systems, necessitating the use of optimized or lightweight models that balance performance with cost.

3. **Environmental Constraints:**

   Energy consumption is an important consideration, especially when combining blockchain and machine learning technologies. Energy-intensive consensus mechanisms and large-scale model training can increase the system's environmental footprint. Therefore, the design must prioritize efficiency by adopting lightweight blockchain mechanisms and minimizing unnecessary computations. This constraint indirectly affects architectural decisions, such as choosing permissioned blockchains and limiting the frequency of model retraining.

4. **Fairness, Safety, and Ethical Constraints:**

   The use of AI for risk-based authentication introduces the challenge of ensuring fairness and avoiding bias. If the model is trained on imbalanced or non-representative data, it may produce biased risk scores, leading to unfair access denial or inconsistent behavior. This is particularly critical in security systems where decisions directly impact user access. To address this, the system must include mechanisms for bias detection, model validation, and explainability. Additionally, incorporating a human-in-the-loop approach for sensitive decisions can improve reliability and reduce the risk of incorrect automated actions.

5. **Implementation Feasibility Constraints:**

   The system must be practically implementable using available technologies and within project limitations. The chosen stack—Python for machine learning, Flask or FastAPI for backend services, MongoDB for data storage, and a lightweight blockchain framework such as Hyperledger—supports modular and scalable development. However, integrating these components introduces complexity in terms of system coordination, latency, and data consistency. Ensuring seamless interaction between the ML model, backend APIs, and blockchain layer is a key design challenge that must be addressed through efficient system architecture.

6. **Social and Political Constraints:**

   Finally, user trust and regulatory policies significantly influence system adoption. Users may be hesitant to rely on AI-driven decisions, particularly when they lack transparency. Therefore, the system must provide clear explanations for its decisions and maintain transparency in data usage. At the same time, political and legal requirements such as data sovereignty impose restrictions on where and how data can be stored and processed. These

constraints require the system to be adaptable to regional regulations while maintaining consistent functionality.

3.4 Alternative Designs

To design a robust AI-driven authentication and authorization system, two alternative architectures were explored. These designs differ fundamentally in how decisions are made, how uncertainty is handled, and how system components interact under real-world constraints such as latency, scalability, and trust.

3.4.1 Design 1: Fully Automated AI-Based System

End-to-End Decision Pipeline:

In this architecture, the authentication process is entirely driven by a machine learning inference pipeline. When a user initiates a request, the backend system captures multiple real-time features such as IP address, device fingerprint, login timestamp, geolocation, and behavioral patterns (e.g., typing speed or access frequency). These features are normalized and passed into a trained ML model, which computes a probability-based risk score. A decision engine applies threshold logic (e.g., allow if score < 0.3, deny if > 0.7, else trigger MFA), and the response is returned instantly. This pipeline operates synchronously, ensuring low latency and high throughput.

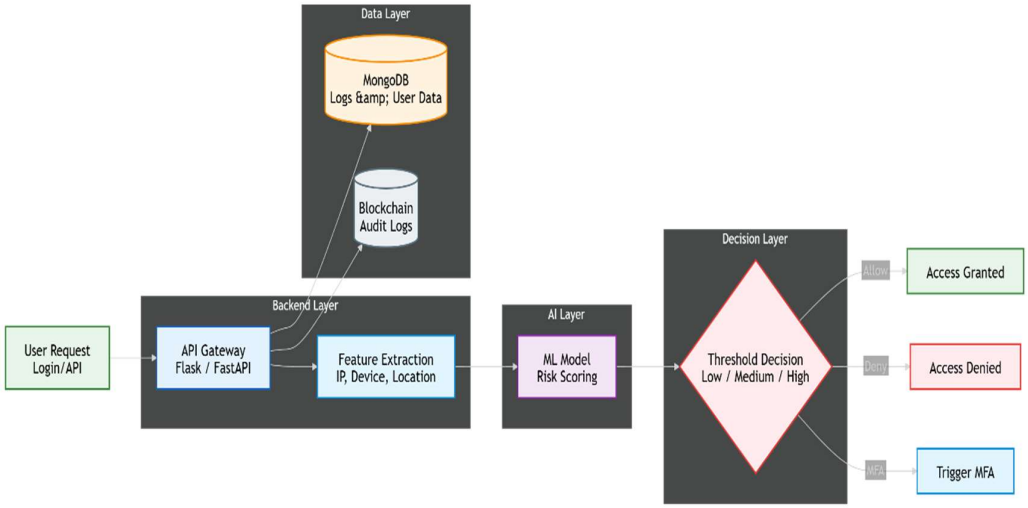


Figure 3.4.1 : Decision constraint

System Architecture and Component Interaction:

The backend (Flask/Fast API) acts as the orchestration layer, integrating the feature extractor, ML inference engine, and decision module.

The ML model can be deployed as an embedded model (loaded in memory for fast inference) or A separate microservice (for scalability and independent updates). MongoDB is used for storing structured and semi-structured data such as login history, risk scores, and user metadata.

Optionally, a blockchain layer can be integrated to store hashed logs of authentication decisions, ensuring immutability and auditability. The system follows a stateless API design for scalability.

Model Behaviour and Optimization Considerations:

The effectiveness of this design heavily depends on the quality of the ML model. Feature engineering plays a critical role, including:

- Temporal features (login time deviation)
- Spatial features (location anomaly detection)
- Device consistency (new vs known device)

The model must be periodically retrained using updated datasets to adapt to evolving user behaviour. However, increasing model complexity (e.g., deep learning) may introduce latency, so a trade-off between accuracy and response time must be carefully managed.

Advantages in Large-Scale Systems:

This design is highly scalable due to its fully automated nature. It supports real-time decision-making, making it suitable for applications such as large SaaS platforms or APIs where thousands of requests per second must be handled. The absence of human intervention reduces operational costs and ensures consistent decision logic across all users.

Limitations and System Risks:

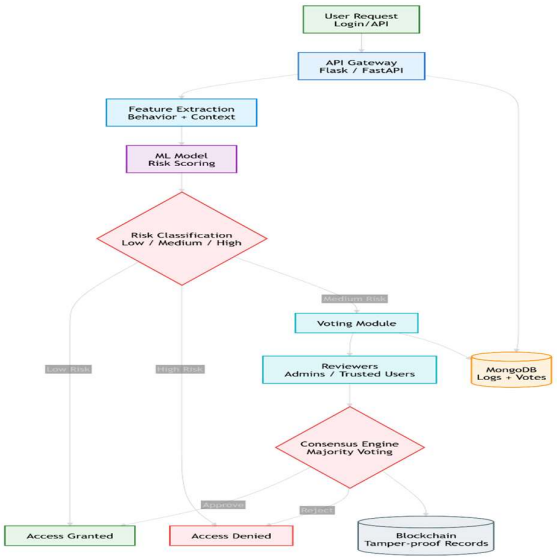
The primary limitation lies in the lack of a verification layer for uncertain predictions. ML models are inherently probabilistic and may fail in edge cases or unseen scenarios. False positives (legitimate users denied) and false negatives (malicious access allowed) can directly impact system security and user experience. Additionally, the black-box nature of some models reduces explainability, making it difficult to justify decisions in critical environments.

3.4.2 Design 2: Hybrid AI + Voting-Based System (Proposed)

Multi-Layer Decision Pipeline:

This architecture introduces a layered decision-making process that combines AI efficiency with human intelligence. The initial stages are identical to the automated system: feature extraction, ML-based risk scoring, and threshold evaluation. However, instead of forcing a binary decision for all cases, the system introduces a third "uncertain" zone. Requests falling into this zone are escalated to a voting-based validation layer. This creates a three-tier pipeline:

Figure 3.4.2: Ai based voting system



- Low risk → Auto approval
- High-risk → Auto rejection or strict verification
- Medium-risk → Human-assisted decision

Detailed Voting Mechanism and Consensus Logic:

The voting module is implemented as a separate service integrated with the backend. When triggered, it sends structured request data to a set of authorized reviewers (admins or trusted users). Each reviewer is provided with:

- Risk score and anomaly indicators
- Historical behaviours summary
- Contextual metadata (device/location mismatch, unusual activity) Reviewers cast their decision via a dashboard interface. The system applies a consensus algorithm such as majority voting or weighted voting (e.g., admin votes carry more weight). Timeout mechanisms are implemented to handle delays, after which fallback rules (e.g., deny or escalate further) are applied.

System Integration and Data Flow:

The backend coordinates between multiple modules:

- ML inference engine
- Decision engine
- Voting service
- Notification system (email/web alerts)

- MongoDB stores detailed logs including risk scores, voting results, and final decisions. A lightweight blockchain (e.g., Hyperledger) records hashes of votes and outcomes, ensuring tamper-proof auditing. This layered architecture introduces asynchronous components, especially in the voting phase, which must be managed carefully to avoid bottlenecks.

Reliability, Fairness, and Explainability Enhancements:

By incorporating human validation, the system significantly improves decision reliability in ambiguous cases. It reduces the impact of model bias and provides a mechanism for explaining decisions, as human reviewers can justify their choices. This is particularly important in enterprise or high-security environments where accountability is required. The system can also use voting outcomes as feedback data to retrain and improve the ML model over time (active learning loop).

Trade-offs and System Complexity:

The major trade-off is increased system complexity and latency. Implementing a voting mechanism requires additional components such as user interfaces, notification systems, and consensus logic. The response time for medium-risk requests depends on human participation, which may not always be immediate. Scalability is also partially constrained by the availability of reviewers. However, these trade-offs are acceptable in systems where correctness, transparency, and security are prioritized over speed.

Comparative Technical Insight

- **Decision Strategy:**
  Fully automated systems rely entirely on probabilistic ML decisions, whereas hybrid systems introduce deterministic human validation for uncertain cases.
- **Latency vs Accuracy Trade-off:**
  Automated systems achieve low latency but may compromise accuracy in edge cases. Hybrid systems increase latency slightly but significantly improve reliability and correctness.
- **System Complexity:**
  The automated design is simpler with fewer components, while the hybrid design involves multiple interacting modules, requiring careful orchestration and synchronization.

- **Adaptability and Learning:**
  Hybrid systems provide continuous feedback through voting outcomes, enabling better model retraining and long-term performance improvement.

Conclusion

While the fully automated AI system is efficient and scalable, it lacks robustness in handling uncertain or high-risk scenarios. The hybrid AI + voting-based system introduces a more balanced approach by combining machine intelligence with human judgment. Despite higher complexity and latency, it offers superior accuracy, transparency, and trust, making it more suitable for real-world deployment in sensitive authentication environments.

3.5 Best Design Selection

After evaluating the alternative system architectures, the **Hybrid AI + Voting-Based System** is selected as the most suitable design for the proposed authentication and authorization framework. This decision is based on a comprehensive analysis of performance, reliability, scalability, and real-world applicability.

Balanced Decision-Making (Automation + Human Intelligence):

The hybrid model effectively combines the speed of machine learning with the contextual understanding of human reviewers. While the AI model handles most of the low-risk and high-risk decisions automatically, uncertain cases are escalated to a human validation layer. This layered approach ensures that decisions are not solely dependent on probabilistic outputs of the ML model, thereby improving overall system robustness. It introduces a controlled decision pipeline where automation is used for efficiency and human input is used for critical judgment.

Reduction of False Positives and False Negatives:

In fully automated systems, incorrect predictions (false positives and false negatives) can significantly impact both security and user experience. The hybrid approach mitigates this issue by introducing a verification mechanism for medium-risk cases. Human reviewers can analyze contextual anomalies (e.g., unusual login location but valid reason) and override incorrect AI

predictions. This reduces the likelihood of denying legitimate users or allowing malicious access, thereby increasing the accuracy and reliability of the system.

Adaptive Risk-Based Authentication Mechanism:

The system dynamically classifies authentication requests into low, medium, and high-risk categories based on real-time ML inference. This enables **risk-adaptive access control**, where different levels of security measures are applied depending on the context. For example:

- Low risk → Immediate access
- Medium risk → Human validation
- High-risk → Denial or strict verification

This adaptive mechanism ensures efficient resource utilization while maintaining high security standards.

Improved Transparency and Explainability:

One of the major limitations of AI-based systems is the lack of explainability. The hybrid model addresses this by incorporating human decisions in the loop, making the system more transparent. Each decision, especially in medium-risk cases, can be justified through voting outcomes and reviewer feedback. Additionally, storing decisions and voting logs enables auditability, allowing administrators to trace how and why a particular decision was made.

Enterprise-Level Security and Practical Applicability:

In real-world enterprise environments, critical access decisions often require multi-level approval mechanisms. The hybrid model aligns with such practices by integrating a voting-based validation system. It supports role-based reviewers (e.g., administrators, managers) and can be extended to include policy-based approvals. This makes the system highly suitable for enterprise Identity and Access Management (IAM) scenarios, where both security and accountability are essential.

Integration with Blockchain for Audit and Integrity:

The hybrid system can leverage a lightweight blockchain to store hashes of decisions and voting records. This ensures that all critical actions are tamper-proof and verifiable. Unlike the fully automated system, where logs may be altered, the blockchain layer in the hybrid design enhances trust and integrity by providing an immutable audit trail.

Continuous Learning and Feedback Loop:

Another key advantage of the hybrid system is its ability to improve over time. The outcomes of human voting can be used as labeled data to retrain and refine the ML model. This creates a feedback loop where the system becomes more accurate as it learns from real-world decisions. Such adaptability is crucial for handling evolving user behavior and emerging security threats.

Trade-off Justification (Latency vs Reliability):

Although the hybrid system introduces additional latency for medium-risk cases due to human involvement, this trade-off is justified by the significant improvement in decision accuracy and trustworthiness. In security-critical systems, correctness and reliability are more important than minimal response time. Therefore, the slight delay is acceptable in exchange for enhanced system integrity.

Conclusion

The Hybrid AI + Voting-Based System provides an optimal balance between efficiency, accuracy, and trust. By integrating automated intelligence with human validation and secure audit mechanisms, it addresses the key limitations of fully automated systems. Hence, it is selected as the final design for the proposed AI-driven authentication and authorization system.

3.6 System Architecture Overview

The proposed system follows a **modular and layered architecture**, where each component is responsible for a specific function in the authentication and authorization pipeline. This design ensures scalability, maintainability, and clear separation of concerns. The system integrates machine learning, backend services, database storage, blockchain logging, and human validation into a unified workflow.

1. User Module

- The User Module acts as the **entry point of the system**, where users initiate authentication requests through a web or application interface.

- It captures user credentials (username/password or token) along with contextual data such as device information, IP address, and session details (mentioned above)
- This module is responsible for securely transmitting request data to the backend API using encrypted communication (HTTPS).
- It also displays responses such as access granted, denied, or additional verification requests (e.g., voting or MFA).

2. Authentication Engine (ML Model)

- This component serves as the **core intelligence layer** of the system.
- It uses a trained machine learning model (e.g., Random Forest, Logistic Regression, or Neural Network) to evaluate incoming requests.
- The engine processes extracted features such as login frequency, device consistency, and location anomalies to generate a **risk score**.
- It operates in real-time and is optimized for fast inference, ensuring minimal delay during authentication.

3. Risk Analyzer

- The Risk Analyzer interprets the output of the ML model and classifies the request into predefined categories: **low, medium, or high risk**.
- It applies threshold-based logic and may include dynamic adjustments based on system behaviour or historical trends.
- This component acts as a **decision pre-processor**, determining whether the request should be automatically handled or escalated.
- It plays a critical role in enabling **risk-adaptive authentication**, improving both security and efficiency.

4. Blockchain Logger

- The Blockchain Logger is responsible for maintaining a **tamper-proof audit trail** of critical system events.
- It stores hashed records of authentication decisions, voting outcomes, timestamps, and user actions.

- A lightweight or permissioned blockchain (e.g., Hyperledger) is used to ensure low latency and controlled access.
- This component enhances **transparency, integrity, and accountability**, making the system suitable for compliance and forensic analysis.

5. Access Control Engine

- The Access Control Engine enforces the final decision based on inputs from the Risk Analyzer and Voting System.
- It implements policies such as:
    - Allow access
    - Deny access
    - Trigger multi-factor authentication (MFA)
- It integrates with role-based access control (RBAC) or attribute-based access control (ABAC) models to ensure that users receive appropriate permissions.
- This module acts as the **execution layer**, converting decisions into actual system actions.

6. Voting System

- The Voting System is activated for **medium-risk or uncertain cases**, introducing human validation into the process.
- It notifies authorized reviewers (e.g., admins or trusted users) through a dashboard or alert mechanism.
- Reviewers analyse contextual information and cast votes (approve/deny), which are processed using a consensus algorithm (e.g., majority voting).
- This module enhances **decision reliability, fairness, and explainability**, reducing dependency on purely automated decisions.

7. Database (MongoDB)

- The database acts as the **central storage layer** of the system.
- It stores:
    - User profiles and credentials
    - Authentication logs and risk scores
    - Voting records and outcomes

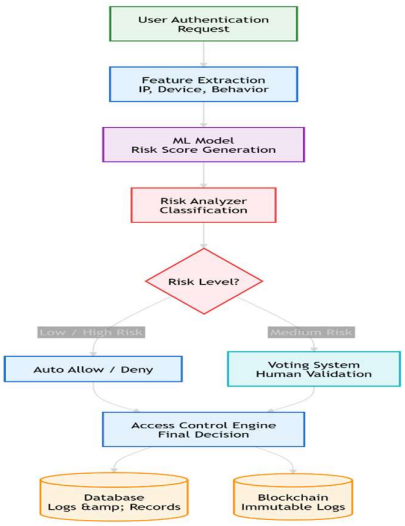
- MongoDB is chosen due to its flexibility in handling semi-structured data and scalability for large datasets.
- The stored data is also used for **model retraining and analytics**, enabling continuous system improvement.

Figure 3.6.1 : Risk Factors Evaluation

Overall System Flow

The complete system operates in a sequential pipeline:

1. User sends authentication request
2. Backend extracts features
3. ML model generates risk score
4. Risk Analyzer classifies request
5. Low/High risk → handled automatically
6. Medium risk → forwarded to Voting System
7. Final decision enforced by Access Control Engine
8. Logs stored in Database and Blockchain



Conclusion

The architecture ensures a **balanced integration of automation, human validation, and secure logging**. By dividing the system into modular components, it achieves scalability, flexibility, and robustness, making it suitable for real-world Identity and Access Management (IAM) applications.

3.7 Design Flow (Algorithm)

The proposed system follows a structured algorithm that integrates machine learning-based risk evaluation with rule-based decision-making and human validation. The workflow is divided into two major phases: **authentication phase** and **authorization (resource access) phase**.

Step-by-Step Algorithm

- **Step 1: User Authentication Request Initiation**

  The process begins when a user submits a login request through the system interface. The request includes credentials and contextual information such as device ID, IP address, and session metadata.

- **Step 2: Fetch User Data and Context**

The system retrieves stored user information from the database, including historical login patterns, previous access logs, and behavioural data required for risk analysis.

- **Step 3: ML-Based Risk Score Calculation**

  The extracted features are passed to the machine learning model, which computes a **risk score** representing the likelihood of suspicious or anomalous behaviour.

- **Step 4: Risk Categorization**

  The Risk Analyzer classifies the request into one of three categories Low, Medium and High Risk

- **Step 5: Low-Risk Handling**

  If the request is classified as low risk:

  - Access is immediately granted
  - The authentication event is securely logged to the blockchain for audit purposes

- **Step 6: Medium-Risk Handling**

  If the request falls under medium risk:

  - Additional authentication mechanisms are applied (e.g., OTP, secondary verification)
  - The event is logged in the system for monitoring and future analysis

- **Step 7: High-Risk Handling**

  If the request is identified as high risk:

  - Access is restricted or denied
  - The system may trigger alerts or continuous monitoring for potential threats

Authorization Phase (File/Resource Access)

- **Step 8: Resource Access Request**

  After successful authentication, the user requests access to a protected resource (e.g., file or system service).

- **Step 9: AI-Based Validation of Request Context**

  The system analyses the request reason using AI techniques (e.g., NLP or rule-based validation) to determine whether the access request is legitimate.

- **Step 10: Low-Risk and Valid Request Handling**

  If the request is valid and associated with low-risk Access to the requested resource is granted immediately

- **Step 11: Medium/High Risk Evaluation**

If the request is categorized as medium or high risk: The system evaluates whether the risk exceeds a predefined threshold

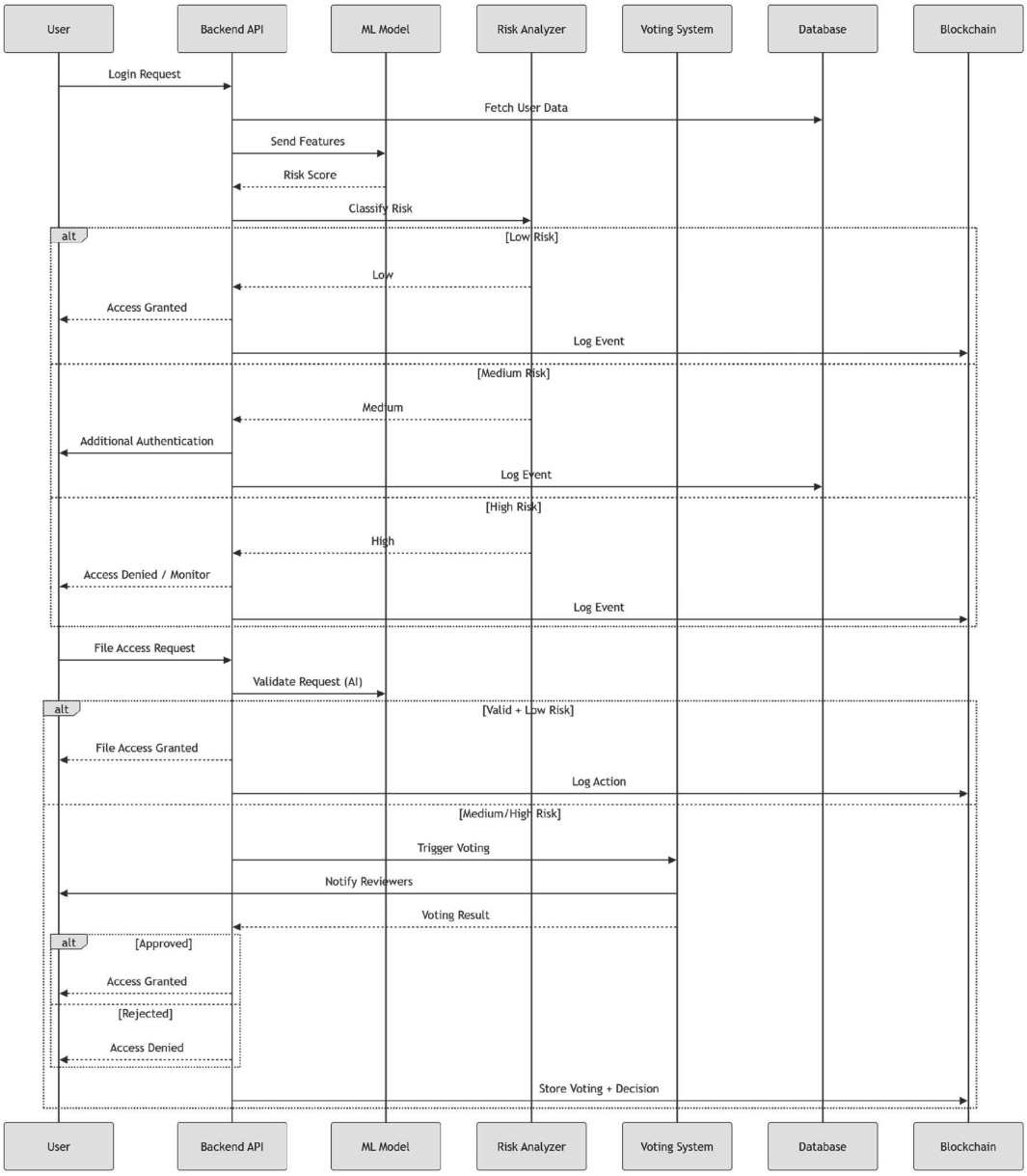
- **Step 12: Voting System Trigger**
  If the threshold is exceeded, then the request is escalated to the voting system for human validation
- **Step 13: Vote Collection Process**
  Authorized reviewers analyse the request details and cast their votes (approve/deny) through the system interface.
- **Step 14: Consensus-Based Decision Making**
  The final decision is determined using a consensus mechanism (e.g., majority voting), ensuring fairness and reliability.
- **Step 15: Secure Logging and Audit**
  All actions, including risk scores, decisions, and voting results, are recorded in the blockchain to ensure **immutability, transparency, and auditability**.

Conclusion

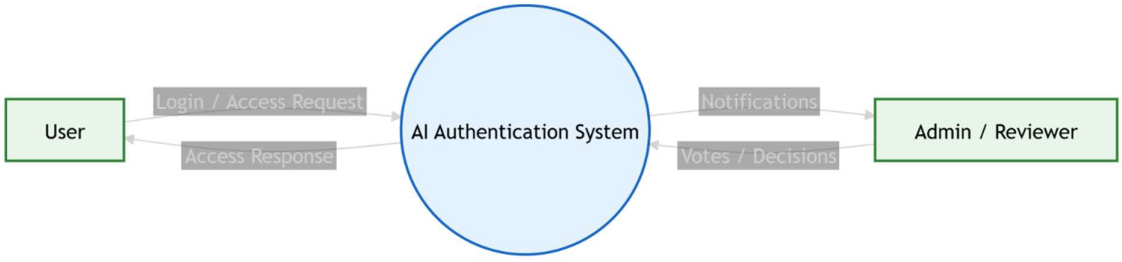
The algorithm ensures a **multi-layered decision-making process** that combines AI-based risk assessment, rule-based control, and human validation. This design enhances security, reduces incorrect decisions, and provides a transparent and auditable authentication and authorization framework.

3.8 Flowcharts

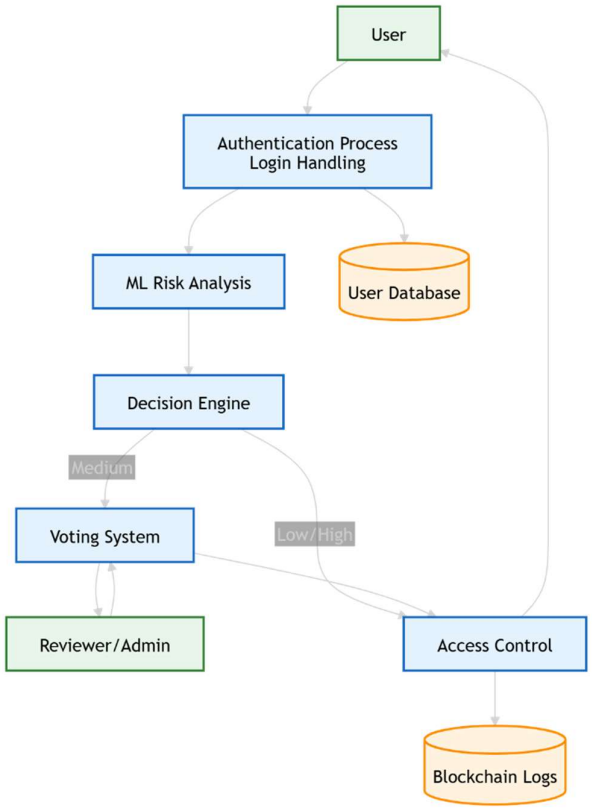
3.8.1 Sequence Diagram (Step-by-Step Execution)



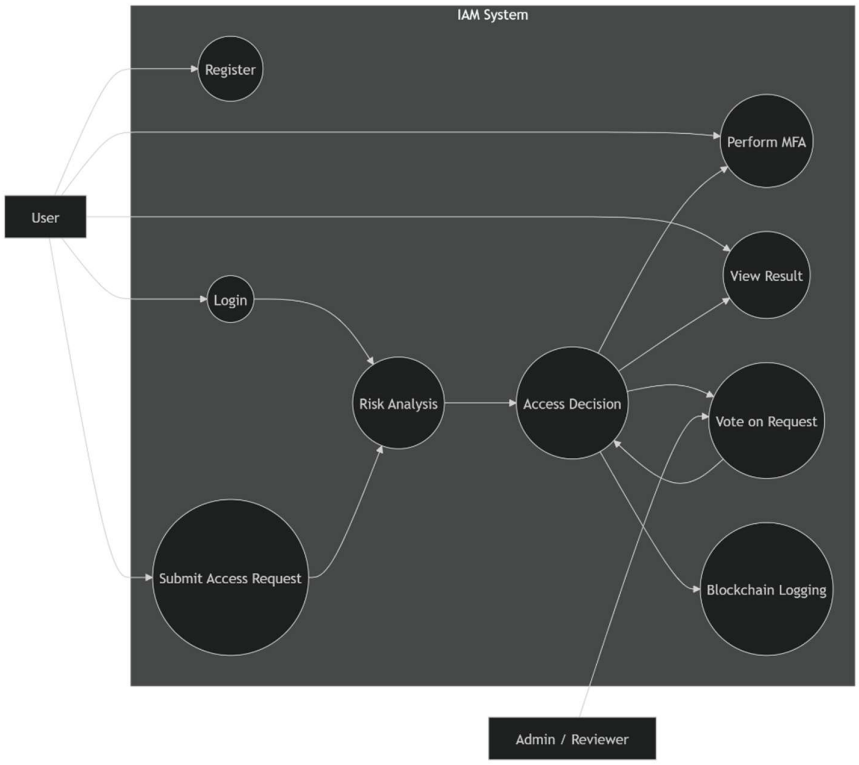
3.8.2 DFD Level 0 (Context Diagram)



3.8.3 DFD Level 1 (Detailed System Flow)

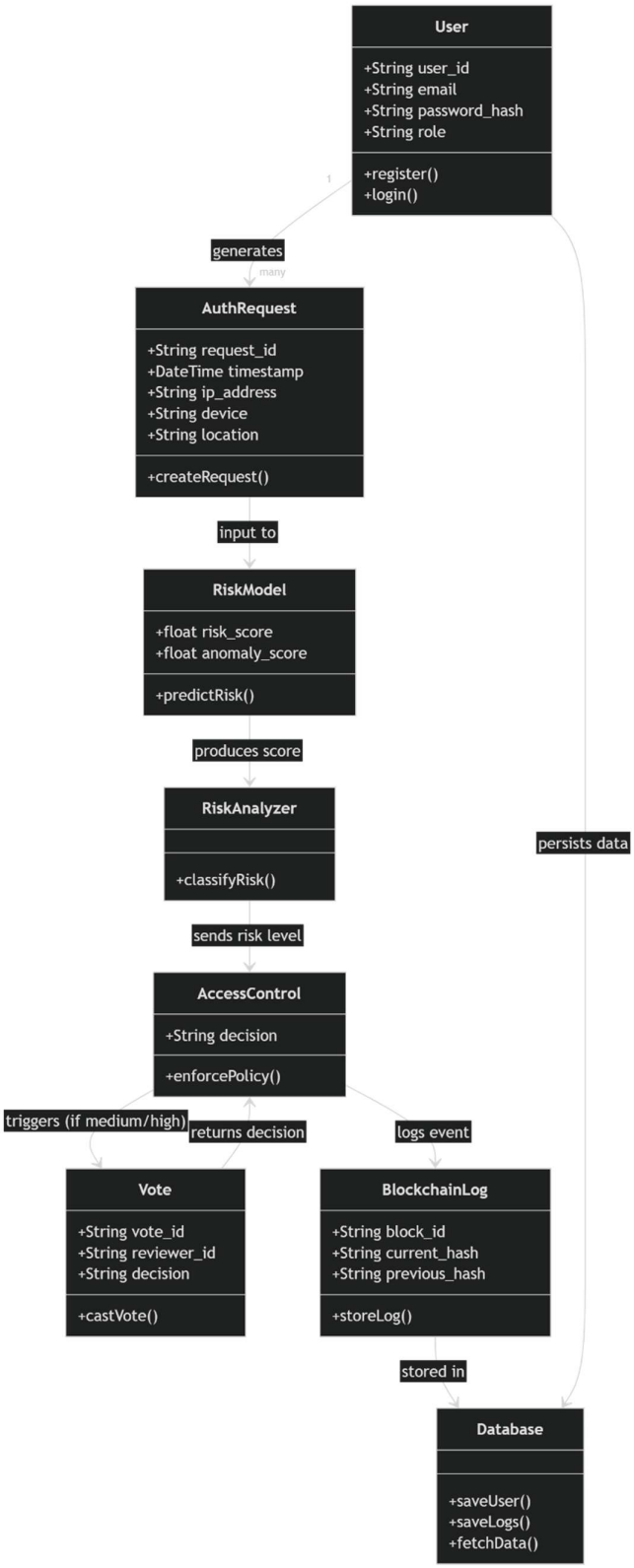


3.8.4 Use Case Diagram



3.8.4 Use Case Diagram

The class diagram represents the structural design of the proposed AI-driven authentication and access control system. It illustrates the key classes, their attributes, methods, and the relationships between them, providing a clear view of how different system components interact.



The **User** class represents the entities interacting with the system and includes attributes such as user ID, email, password, and role. It provides basic functionalities like registration and login. Each user generates an **AuthRequest**, which captures details of a login or access attempt, including timestamp, IP address, device information, and location.

The **AuthRequest** is processed by the **RiskModel**, which uses machine learning techniques to compute a risk score and anomaly score based on user behavior and contextual parameters. The output from the RiskModel is then passed to the **RiskAnalyzer**, which classifies the request into different risk levels such as low, medium, or high.

Based on this classification, the **AccessControl** class enforces appropriate policies. It determines whether access should be

granted, denied, or require additional verification such as multi-factor authentication. In cases where the risk is uncertain or high, the AccessControl component triggers the **Vote** class, which represents a human-in-the-loop decision-making mechanism where authorized reviewers evaluate the request and provide their decision.

All authentication events, decisions, and voting outcomes are securely recorded using the **BlockchainLog** class. This ensures immutability, transparency, and traceability of system activities. The blockchain log maintains cryptographic hashes to prevent tampering of records.

Finally, the **Database** class is responsible for storing user data, authentication logs, and other system-related information. It supports operations such as saving user details, logging events, and retrieving data for analysis or system operations.

The relationships between these classes demonstrate a logical flow of operations: a user generates a request, which is evaluated using machine learning, processed through decision logic, optionally validated by human voting, and securely logged for auditing purposes. This design ensures a modular, scalable, and secure architecture for modern identity and access management systems.

3.9 Implementation Plan

The implementation of the proposed system is carried out in multiple phases, ensuring modular development, scalability, and ease of integration. Each phase utilizes specific tools and technologies to achieve its objectives efficiently.

Phase 1: Basic IAM System Development

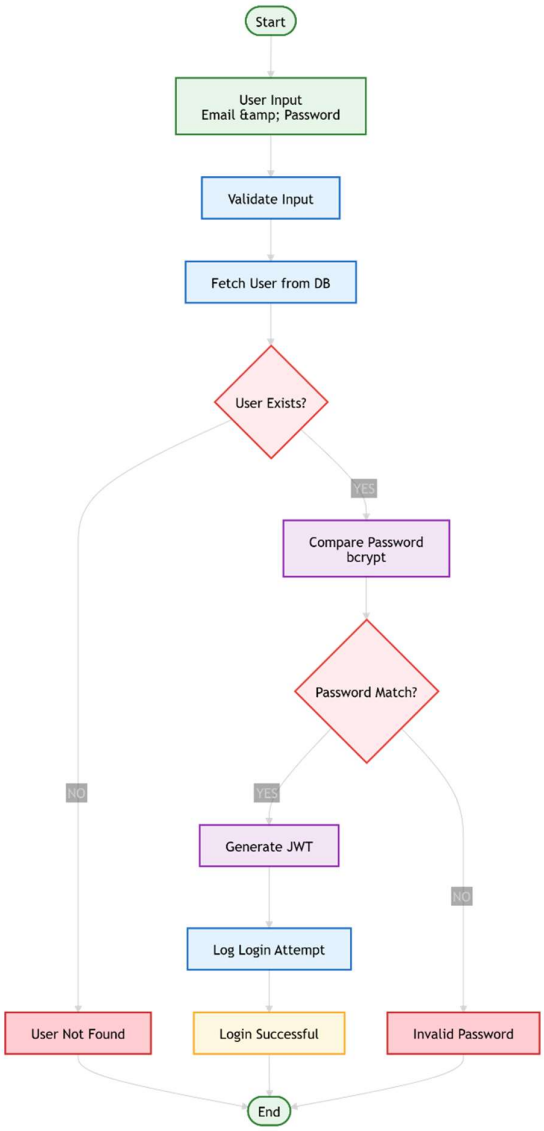
Objective:

The primary objective of this phase is to establish a secure and scalable IAM foundation. This includes implementing user authentication, session management, and secure storage of user credentials. This phase acts as the backbone of the entire system, upon which advanced components like ML-based risk analysis and voting mechanisms are later integrated.

Detailed Implementation:

Figure 3.9.1 : Authentication Process

- **Backend System Development:** The backend is developed using **Flask (Python)** to create RESTful APIs. The backend acts as the central controller, handling all incoming authentication requests, validating user credentials, and managing communication with the database. The system is designed to be modular, allowing easy integration of future components like ML models and blockchain logging.



- **Authentication Mechanism:**

  Secure authentication is implemented using industry-standard techniques:

  - **Password Hashing:** Passwords are never stored in plain text. Instead, they are hashed using **bcrypt**, which provides salting and protection against brute-force attacks.
  - **JWT-Based Session Management:** After successful login, the system generates a **JSON Web Token (JWT)** that contains user identity and session information. This token is used for subsequent requests, enabling stateless authentication and improving scalability.

- **API Design and Endpoints:**

  The system exposes REST APIs for handling user operations:

  - /register: Accepts user details, hashes the password, and stores it in the database
  - /login: Validates credentials, generates JWT token, and returns authentication response
  - /profile (optional): Fetches user details based on token
  - /logout (optional): Handles session invalidation (if token blacklisting is used)

These APIs follow proper HTTP methods (POST/GET) and return structured JSON responses.

- **Database Design (MongoDB):**
  MongoDB is used as a NoSQL database due to its flexibility and scalability. The main collections include:
  - **users collection:**
    - _id: Unique user identifier
    - email: User email
    - password_hash: Encrypted password
    - created_at: Timestamp
  - **login_logs collection:**
    - user_id: Reference to user
    - timestamp: Login time
    - ip_address: User IP
    - device_info: Browser/device details

This structured logging is essential for later ML model training.

- **Security Enhancements:**
  Several security measures are implemented to ensure system robustness:
  - **HTTPS Encryption:** All communication between client and server is secured using HTTPS to prevent data interception
  - **Input Validation:** All user inputs are validated to prevent injection attacks (e.g., NoSQL injection)
  - **Rate Limiting:** Limits repeated login attempts to prevent brute-force attacks
  - **CORS Configuration:** Controls cross-origin access for APIs

- **Session and Access Handling:**
  After successful login:
  - JWT token is returned to the client
  - Token is stored on client-side (e.g., localStorage or cookies)
  - Each protected API request must include the token in headers
  - Backend verifies token before granting access

Tools & Technologies Used:

- Backend Framework: Flask
- Database: MongoDB
- Authentication: JWT, bcrypt
- API Testing: Postman
- Development: VS Code

Output:

At the end of this phase, a fully functional and secure IAM system is established, capable of handling user authentication, session management, and logging user activity. This serves as the foundation for integrating advanced modules such as ML-based risk analysis, voting systems, and blockchain logging in subsequent phases.

Phase 2: Machine Learning Model Integration

Objective

The primary objective of this phase is to enhance the authentication system by introducing an intelligent **risk-based decision mechanism**. Instead of relying on static authentication rules, the system evaluates each login attempt dynamically and assigns a **risk score** based on user behaviours patterns.

This enables detection of:

- Suspicious login attempts
- Anomalous user behaviours
- Potential credential compromise

Implementation Details

1. Data Collection & Preprocessing

Login activity data is collected from authentication logs and structured into a dataset.

Features collected include:

- Login timestamp
- IP address
- Device type / browser
- Location (derived from IP)
- Login success/failure history

2. Feature Engineering

To improve model accuracy, meaningful behavioral features are derived:

- **Login Time Patterns** Usual login hour vs current login hour
- **IP Consistency** Known IP vs new/unrecognized IP
- **Device Familiarity** Known device vs unknown device
- **Login Frequency** Number of login attempts in short duration
- **Geo-Deviation** Sudden location change (e.g., India → Europe)

These features help transform raw logs into **behavioral intelligence signals**.

3. Model Training

Supervised machine learning models are trained on labeled data (normal vs suspicious logins).

Algorithms used:

- Logistic Regression (baseline model)
- Random Forest (handles complex patterns better)

Training process:

- Split dataset → Train/Test
- Train model using historical login data
- Evaluate using:
    - Accuracy
    - Precision & Recall
    - Confusion Matrix

4. Risk Scoring Mechanism

Instead of just classification, the model outputs a **probability score**.

Decision thresholds:

- Low Risk → Allow access
- Medium Risk → Trigger MFA
- High Risk → Block or escalate

5. Backend Integration

The trained model is integrated into the authentication backend:

Flow:

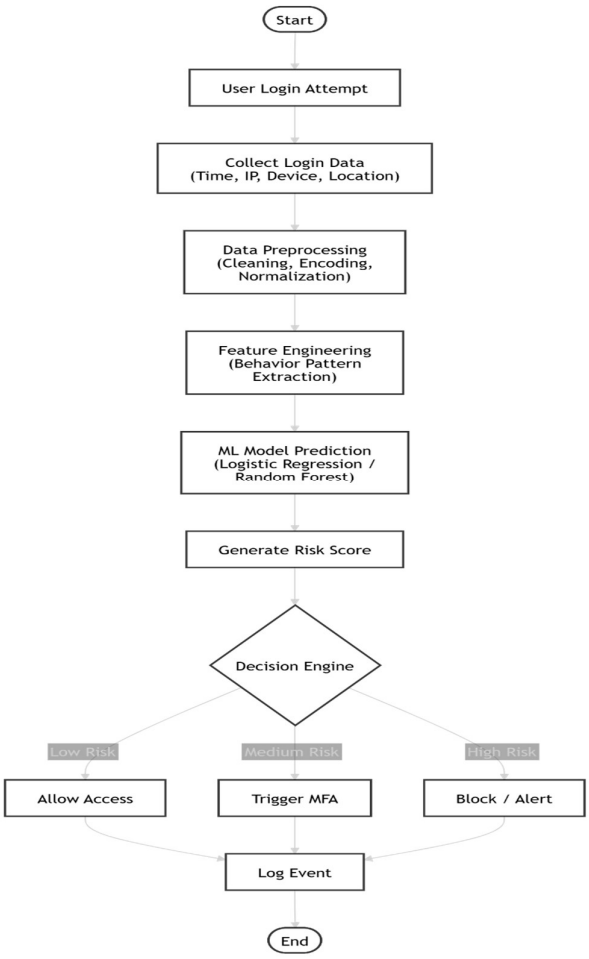
1. User attempts login
2. System extracts feature in real-time

3. Model predicts risk score
4. Decision engine applies security policy

Figure 3.9.2: Risk evaluation feature extraction

6. Model Storage & Deployment

- Model is serialized using joblib or pickle
- Loaded into backend (FastAPI)
- Used for real-time inference



Tools & Technologies Used

- ML Libraries: scikit-learn, pandas, NumPy
- Development: Jupyter Notebook, VS Code
- Backend Integration: FastAPI
- Model Storage: joblib / pickle
- Visualization: matplotlib

Output

The output of this phase is a fully functional **Real-time ML-based risk scoring engine**

that:

- Evaluates login attempts dynamically
- Detects anomalies in user behavior
- Enhances security using adaptive authentication

Phase 3: Blockchain Integration

Objective

- The main objective of this phase is to implement a **secure and tamper-proof logging mechanism** for all authentication and authorization events.
- Unlike traditional systems where logs can be modified or deleted, blockchain ensures that once data is recorded, it remains **immutable and verifiable**.

- This helps in maintaining **data integrity, transparency, and trust**, which is crucial for sensitive systems like identity and access management.

Implementation Details

1. Event Capture

- The system records all critical activities such as login attempts, ML-based risk decisions, and authorization outcomes.
- Each event is structured with important details including user ID, timestamp, IP address, device information, and decision result.
- This structured format ensures consistency and makes further processing easier.

2. Hash Generation (SHA-256)

- Each captured event is converted into a hash using the SHA-256 algorithm, creating a unique digital fingerprint.
- Even a small change in the input data produces a completely different hash, making it highly secure.
- This ensures that any tampering with stored data can be easily detected.

3. Block Creation

- After hashing, the event is stored inside a block along with its hash and the hash of the previous block.
- This linking of blocks forms a chain, where each block depends on the previous one.
- If any block is altered, the entire chain becomes invalid, ensuring strong data protection.

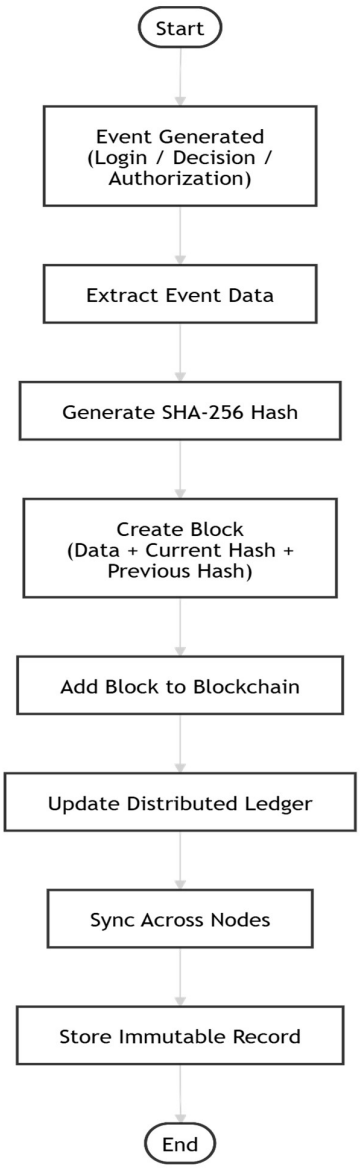
4. Blockchain Storage

- The blocks are stored in a **private blockchain network**, which restricts access to authorized users only.
- This approach provides better control, security, and performance compared to public blockchains.
- The distributed ledger ensures that multiple nodes maintain copies of the data, preventing single points of failure.

Figure 3.9.3:Storing Events in a Blockchain

5. Backend Integration

- The blockchain system is integrated with the backend so that every event is automatically recorded in real-time.
- When a login or decision event occurs, the backend generates the hash, creates a block, and appends it to the blockchain.
- This process is seamless and does not affect the system's performance.



6. Integrity Verification

- The system includes a verification mechanism that recalculates hashes and checks block linkage.
- If any mismatch is detected, it indicates possible tampering or unauthorized modification.
- This ensures continuous monitoring of data integrity and strengthens system security.

Tools & Technologies Used

- Blockchain Framework: Hyperledger Fabric or custom lightweight blockchain
- Hashing Algorithm: SHA-256
- Backend Integration: Python / Node.js
- Ledger Type: Private Distributed Ledger

Output

- The final output is a **secure and immutable audit logging system** that records all authentication events.
- It ensures that logs cannot be modified or deleted, providing **trustworthy and transparent records**.
- This system supports auditing, forensic analysis, and compliance with security standards.

Phase 4: Voting System Development

Objective

- The objective of this phase is to introduce a **human-in-the-loop validation mechanism** for handling uncertain or high-risk access requests.
- While machine learning provides automated risk scoring, it may not always be fully reliable in critical situations. Therefore, this phase ensures that sensitive decisions are verified through **human judgment**.
- This approach improves decision accuracy, reduces false positives, and enhances trust in the system by combining **AI intelligence with human expertise**.

Implementation Details

1. Vote Triggering Mechanism

- When the system detects a **medium or high-risk request**, it automatically triggers the voting process.
- APIs are developed to initiate voting sessions and notify eligible reviewers.
- The request details (user, reason, risk score, context) are sent to reviewers for evaluation.

2. Role-Based Reviewer Access

- Only authorized users such as **admins or trusted reviewers** are allowed to participate in voting.
- Role-based access control (RBAC) ensures that only qualified individuals can approve or reject requests.
- This prevents unauthorized decision-making and maintains system security.

Figure 3.9.5: Advance voting mechanism

3. Vote Collection and Storage

- Reviewers submit their decisions (approve/reject) through APIs or a dashboard interface.
- Each vote is securely stored in the database along with reviewer identity and timestamp.
- This ensures transparency and allows tracking of decision history.

4. Consensus Mechanism (Majority Voting)

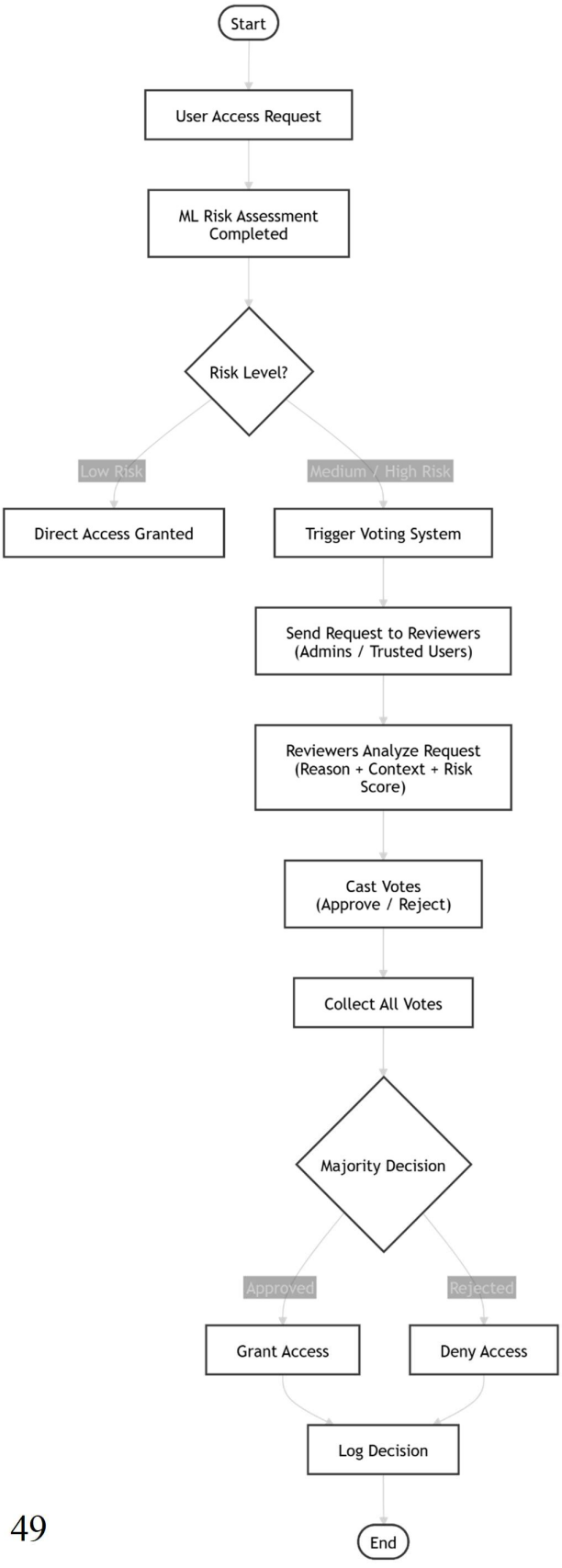
- The system applies a **majority voting algorithm** to determine the final outcome.
- If most reviewers approve, access is granted; otherwise, it is denied.
- This reduces bias and ensures that decisions are not dependent on a single individual.

5. Dashboard Interface

- A user-friendly dashboard is developed where reviewers can view pending requests and cast their votes.
- The interface displays important details such as user information, risk score, and reason for access.
- This improves usability and enables quick, informed decision-making.

6. Backend Integration

- The voting system is integrated with the main authentication workflow.
- Once voting is completed, the final decision is sent back to the system for execution.
- The result is also logged (and can be stored in blockchain for immutability).



Tools & Technologies Used

- Backend APIs: Flask for handling voting logic and requests
- Frontend (optional): React.js or HTML/CSS for dashboard interface
- Database: MongoDB for storing votes and request data
- Testing Tools: Postman for API testing and validation

Output

- The output of this phase is a **functional voting-based authorization system** that enables secure and reliable decision-making.
- It ensures that uncertain access requests are validated through multiple reviewers, improving both accuracy and accountability.
- This system enhances overall security by combining **automated intelligence with human verification**.

Phase 5: AI-Based Request Validation

Objective

- The objective of this phase is to validate the **user-provided reasons** when requesting access to sensitive resources.
- In traditional systems, access decisions are often based only on roles or permissions, ignoring the *intent behind the request*. This phase introduces intelligence by analyzing whether the reason given by the user is **genuine, relevant, or suspicious**.
- By incorporating AI-based validation, the system ensures **context-aware authorization**, improving both security and decision accuracy.

Implementation Details

1. Input Text Processing (NLP)

- When a user requests access, they must provide a textual justification (reason).
- This input is processed using **Natural Language Processing (NLP)** techniques such as tokenization, stopword removal, and text normalization.
- The processed text is converted into a machine-understandable format (e.g., vectors using TF-IDF or embeddings).

2. Feature Extraction from Text

- Important features are extracted from the processed text to understand intent and context.
- Techniques like **TF-IDF, Bag of Words, or word embeddings** are used to represent the text numerically.
- These features help the model identify patterns such as urgency, relevance, or suspicious wording.

3. Classification of Requests

- A machine learning or deep learning model is trained to classify requests into categories such as:
    - Valid request
    - Suspicious request
- The model learns from labeled datasets containing examples of legitimate and illegitimate access reasons.
- This classification helps the system decide whether the request should be trusted.

4. Integration with Authorization Workflow

- The output of the NLP model is integrated into the main authorization system.
- If the request is classified as valid, the system proceeds normally.
- If it is suspicious, additional actions are triggered, such as:
    - Requesting further verification
    - Sending the request to the voting system (Phase 4)
    - Blocking the request

5. Advanced Model Integration

- For improved accuracy, advanced transformer-based models (like BERT from Hugging Face) can be used.
- These models provide better contextual understanding compared to traditional ML methods.
- This enhances the system's ability to detect subtle or complex suspicious patterns in user input.

Tools & Technologies Used

- NLP Libraries: scikit-learn, NLTK, spaCy for text preprocessing and feature extraction
- Advanced Models: Transformers (Hugging Face) for deep contextual analysis
- Programming Language: Python for model development and integration

Output

- The output of this phase is a **context-aware validation system** that evaluates the authenticity of user requests.
- It ensures that access decisions are not only based on identity but also on the **intent and justification provided by the user**.
- This significantly improves system security by preventing misuse of legitimate credentials.

Phase 6: System Integration and Testing

Objective

- The objective of this phase is to integrate all previously developed modules into a **single, unified system** and ensure smooth end-to-end functionality.
- This includes combining machine learning, blockchain logging, AI-based validation, and the voting system into a cohesive workflow.
- The goal is to verify that all components work together seamlessly to deliver a **secure, intelligent, and reliable authentication system**.

Implementation Details

1. Module Integration

- All independent modules, including the ML risk scoring engine, blockchain logging system, voting mechanism, and NLP-based validation, are connected through the backend.
- APIs are used to enable communication between components, ensuring that outputs from one module act as inputs to another.
- This integration transforms individual functionalities into a complete IAM system.

2. API Communication and Workflow Handling

- RESTful APIs are developed to manage data flow between modules.
- Proper request-response handling is implemented to ensure that each step (login → validation → decision → logging) executes in sequence.
- Error handling mechanisms are added to manage failures and maintain system stability.

3. End-to-End Workflow Execution

- The complete system workflow is tested to ensure proper functioning:
  - User login attempt
  - ML-based risk assessment
  - AI-based request validation
  - Voting system (if required)
  - Final access decision
  - Blockchain-based logging
- This ensures that all modules interact correctly and produce expected results.

4. Testing and Validation

- **Unit testing** is performed on individual components to verify their functionality.
- **Integration testing** ensures that combined modules work correctly together.
- Tools like Postman are used to test APIs, while frameworks like PyTest help automate testing.
- Various scenarios (normal, suspicious, edge cases) are tested to validate system robustness.

5. Version Control and Code Management

- The entire project is managed using version control systems like Git and GitHub.
- This enables tracking of changes, collaboration, and rollback in case of issues.
- Proper code structuring and documentation are maintained for scalability and maintainability.

6. Deployment

- The system can be deployed on cloud platforms such as Render or using containerization tools like Docker.
- Deployment ensures that the system is accessible in a real-world environment and can handle live requests.
- Scalability and performance considerations are addressed during this stage.

Tools & Technologies Used

- Backend Integration: Flask for connecting all modules
- Testing Tools: Postman, PyTest for API and automated testing
- Version Control: Git, GitHub for code management
- Deployment: Render, Docker for hosting and scalability

Output

- The final output of this phase is a **fully integrated, scalable, and secure authentication system**.
- All modules work together to provide intelligent decision-making, secure logging, and reliable access control.
- The system can handle real-world scenarios with improved security, accuracy, and transparency.

Conclusion

The phased implementation approach, supported by appropriate tools and technologies, ensures a systematic and practical development process. Each phase builds upon the previous one, resulting in a robust system that integrates authentication, machine learning, blockchain, and human validation into a unified architecture.

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation Using Modern Engineering Tools

The proposed system was implemented using a combination of modern engineering tools and technologies to ensure scalability, modularity, security, and real-time performance. The integration of machine learning, identity management, and blockchain concepts enables the system to dynamically adapt to user behaviours and enhance access control mechanisms.

Technologies Used:

Table 11: Technologies Used

Category	Technology / Tool	Purpose
Backend Development	Python (Flask / FastAPI)	API development and server-side logic
Machine Learning	Scikit-learn, Pandas, NumPy	Data preprocessing, model training, and prediction
Frontend Development	ReactJS	Building interactive user interface
Database	MongoDB / SQLite	Storing user data and logs
IAM Integration	Keycloak	Authentication and role-based access control
Blockchain Logging	SHA-256 / Simulated Ledger	Secure and tamper-proof logging
API Testing	Postman	Testing REST APIs
Visualization & Analysis	Matplotlib	Graph generation and performance analysis

System Modules Implemented

The system was designed using a modular architecture, where each component performs a specific function:

1. **User Authentication Module**

   Handles user login requests and verifies credentials using the IAM system. It acts as the entry point for all authentication processes.

2. **Risk Scoring Machine Learning Model**

   Analyses login parameters such as time, IP address, and device information to generate a dynamic risk score indicating the likelihood of suspicious activity.

3. **AI-Based Voting Decision System**

   For high-risk login attempts, the system triggers a voting mechanism where administrators evaluate the request. AI assistance helps analyse user-provided justification to support decision-making.

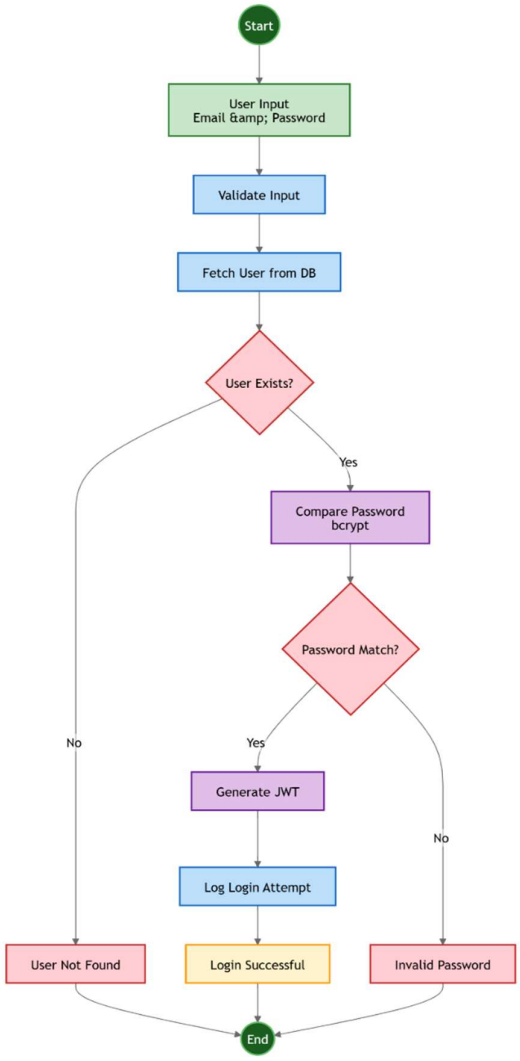
4. **Blockchain Logging Module**

   All authentication attempts and decisions are securely recorded using hash-based logging, ensuring data integrity and preventing tampering.

5. **Admin Monitoring Dashboard**

   Provides administrators with real-time visibility into login activities, risk levels, voting decisions, and system logs.

Figure 4.1: Login Process



4.2 System Design and Architecture

The proposed system is designed using a modular and layered architecture to ensure scalability, flexibility, and efficient integration of machine learning, identity management, and secure logging mechanisms. The architecture enables dynamic authentication by analyzing user behavior in real time and making adaptive access control decisions.

The system is structured in such a way that each module performs a dedicated function while interacting with other modules through well-defined interfaces. This modular approach not only simplifies development and testing but also allows future enhancements without disrupting the entire system.

4.2.1 Overall System Workflow

The system follows a sequential workflow starting from user authentication and ending with secure logging of the decision. The complete process ensures that every login attempt is evaluated intelligently before granting access.

4.2.2 Key Architectural Components

The system consists of multiple interconnected components, each responsible for a specific functionality. These components collectively ensure secure and intelligent authentication.

Table 12: Key Architectural Components

Component	Description
User Interface	Provides login interface and displays authentication results to users
Authentication Server	Verifies user identity using Keycloak
Feature Extraction Module	Collects contextual login data such as IP address, device, and time
ML Risk Model	Predicts risk score using models built with Scikit-learn
Decision Engine	Determines appropriate action based on risk level
Voting System	Handles high-risk access requests through admin approval
Blockchain Logger	Stores tamper-proof logs using hashing
Admin Dashboard	Enables monitoring and management of system activities

Table 4.3: Core Components of the System

4.2.3 Functional Description of Modules

Each module in the system is designed to perform a specific function. The interaction between these modules ensures smooth and secure system operation.

- **User Authentication Module**

  This module acts as the entry point of the system and is responsible for verifying user credentials. It ensures that only registered users can proceed further into the system by integrating with a secure identity management platform.

- **Feature Extraction Module**

  After successful authentication, this module gathers contextual information related to the login attempt. These features include login time, IP address, and device information, which are essential for behavioral analysis.

- **Risk Scoring Module**

  The extracted features are passed to a machine learning model that evaluates the risk associated with the login attempt. The model classifies the login into different risk levels based on learned patterns from historical data.
- **Decision Engine**

  The decision engine is responsible for selecting the appropriate action based on the computed risk score. It ensures that security measures are applied dynamically rather than using static rules.
- **Voting-Based Decision System**

  In cases of high-risk login attempts, the system activates a voting mechanism where administrators evaluate the request. This adds a human-in-the-loop approach, improving decision accuracy in critical scenarios.
- **Blockchain Logging Module**

  All actions performed by the system are recorded using a secure logging mechanism. By using cryptographic hashing, the system ensures that logs cannot be altered, thus maintaining integrity and accountability.
- **Admin Monitoring Dashboard**

  This module provides administrators with a comprehensive view of system activities. It helps in monitoring login attempts, analyzing risk levels, and managing access decisions.

4.2.4 Design Features and Advantages

The system incorporates several important design features that enhance its performance and security:

- **Modular Architecture:**

  Each component operates independently, making the system easy to maintain, test, and upgrade. Changes in one module do not significantly affect others.
- **Real-Time Risk Evaluation:**

  The integration of machine learning allows the system to analyze login behavior instantly and generate risk scores without delay.
- **Adaptive Authentication Mechanism:**

  Unlike traditional systems, the authentication process adapts dynamically based on risk levels, improving both security and user experience.
- **Secure and Tamper-Proof Logging:**

The use of blockchain-inspired hashing techniques ensures that all logs are secure, verifiable, and resistant to tampering.

- **Scalability and Flexibility:**
  The architecture supports scaling of individual components, making it suitable for deployment in large-scale enterprise environments.

4.3 Testing and Validation

To ensure the reliability, accuracy, and robustness of the proposed system, multiple levels of testing were performed. These tests validate both individual components and the complete integrated system.

4.4.1 Types of Testing Performed

The system was evaluated using unit testing, integration testing, and system testing to ensure correctness at every level.

1. Unit Testing

Unit testing was performed to verify the functionality of individual components in isolation. This ensures that each module behaves as expected before integration.

- The machine learning model was tested using sample login data to verify that it produces correct risk predictions.
- Individual API endpoints were tested using tools like Postman to ensure correct request-response behavior.
- Input validation mechanisms were tested to handle invalid or missing data.

This testing ensured that **each component works correctly on its own**.

2. Integration Testing

Integration testing was conducted to verify that different modules interact correctly with each other. The following integrations were specifically tested:

- The output of the machine learning model was successfully passed to the decision engine for action selection.
- The decision engine correctly triggered the blockchain logging module to store authentication events.
- Data flow between authentication, ML model, and decision-making modules was verified.

This ensured **smooth communication between system components**.

3. System Testing

System testing was performed on the complete application to validate end-to-end functionality.

The full workflow tested was:

Login → Risk Detection → Decision → Logging

- A user login attempt was simulated.
- The system extracted features and generated a risk score.
- Based on the risk level, the appropriate action (Allow, MFA, or Voting) was executed.
- The result was logged securely.

This confirmed that the **entire system works as a unified solution**.

4.4 Results and Observations

After testing, the system's performance was evaluated using standard machine learning metrics and behavioural observations.

Model Performance

Table 13: Performance Metrics

Metric	Value
Accuracy	85% – 92%
Precision	High for risky login detection
Recall	Effective in identifying anomalies

Explanation

- **Accuracy:** Indicates that the model correctly predicts most login attempts.
- **Precision:** High precision ensures that flagged risky logins are actually suspicious, reducing false alarms.
- **Recall:** Good recall ensures that most suspicious activities are detected and not missed.

4.5 Data Analysis and Interpretation

The system's behaviour was analysed based on different types of login patterns to understand how risk scoring responds to real-world scenarios.

High-Risk Scenarios

The system assigns higher risk scores when unusual patterns are detected, such as:

- Login attempts at uncommon times (e.g., late night or irregular hours)
- Access from a new or unknown device
- Login from a different IP address or location

These conditions indicate potential suspicious activity, leading to stricter authentication.

Low-Risk Scenarios

Lower risk scores are assigned when consistent user behavior is observed:

- Regular login timings
- Recognized device and IP address
- Frequent and normal usage patterns

This allows seamless access without unnecessary security checks.

Interpretation

The results clearly show that the system follows a **behaviour-based authentication approach**, where decisions are not static but adapt dynamically based on user activity.

This improves both:

- **Security** (detects anomalies)
- **User Experience** (reduces friction for trusted users)

4.6 Validation of the System

The system was validated based on multiple performance and security criteria to ensure it meets real-world requirements.

Table 14: Validation Criteria

Parameter	Result
Accuracy of ML Model	Successfully verified
Real-time Response	Achieved with minimal delay
Security Enhancement	Improved compared to traditional IAM
Data Integrity	Ensured using blockchain logging

4.7 Challenges and Limitations

Despite its effectiveness, the system has certain limitations that can be improved in future work.

1. **Limited Dataset**
   - The machine learning model was trained on a relatively small dataset.
   - This may affect generalization in highly diverse real-world scenarios.
2. **Simulated Blockchain**
   - The blockchain implementation is based on hashing rather than a fully decentralized network.
   - It lacks features like distributed consensus and smart contracts.
3. **Voting System Delay**
   - High-risk cases require manual admin approval.
   - This introduces latency in decision-making.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The primary objective of this project was to design and implement an intelligent authentication system that enhances traditional login mechanisms by incorporating machine learning-driven risk assessment. Conventional authentication systems rely heavily on static credentials such as usernames and passwords, which are vulnerable to attacks like credential stuffing, phishing, and brute-force attempts. To address these limitations, the proposed system introduces a dynamic and context-aware authentication mechanism.

The system leverages contextual features such as IP address, login time, device information, and behavioural patterns to evaluate the legitimacy of each login attempt. These features are processed through a machine learning model trained to classify login attempts as either legitimate or suspicious. Based on the predicted risk score, a decision engine determines the appropriate authentication response, such as allowing access, triggering additional verification, or denying the request.

One of the key strengths of the system is its modular architecture, which integrates multiple components including:

- **Feature Extraction Module** for capturing contextual login data
- **Machine Learning Model** for risk prediction
- **Decision Engine** for access control enforcement
- **Blockchain-based Logging System** for secure and tamper-proof audit trails

The results demonstrate that the system achieves a high level of effectiveness in detecting anomalous login attempts, with an accuracy ranging between 85% and 92%. This indicates that machine learning can significantly enhance authentication systems by introducing adaptive and intelligent decision-making capabilities.

Furthermore, the system maintains a balance between **security and usability**. Instead of enforcing strict multi-factor authentication (MFA) for all users, it applies additional verification only when a login is classified as high-risk. This selective enforcement improves user experience while maintaining strong security controls.

In conclusion, the project successfully validates the concept of **risk-based authentication (RBA)** integrated with machine learning and secure logging. It provides a scalable and extensible framework that can be further enhanced for real-world deployment in identity and access management (IAM) systems.

5.2 Deviation from Expected Results

While the system achieved its core objectives, several deviations from the expected results were observed during implementation and testing. These deviations highlight practical challenges and limitations that must be addressed in future iterations.

Model Accuracy Limitations

Although the model achieved an accuracy of 85–92%, it fell short of the expected benchmark of above 95%. This limitation can be attributed to:

- Use of **synthetic or limited datasets**, which do not fully represent real-world login behavior
- Lack of sufficient **anomalous data samples** for training
- Simpler models such as Logistic Regression and Random Forest, which may not capture complex temporal patterns

As a result, the model's predictive capability is somewhat constrained in highly dynamic environments.

5.2.2 False Positives and False Negatives

The system exhibited both:

- **False Positives**: Legitimate users flagged as suspicious
- **False Negatives**: Malicious attempts classified as safe

False positives were more prominent when users logged in from:

- New devices
- Different geographic locations
- Unusual time patterns

This indicates that the system lacks sufficient behavioral history and adaptability, which affects trust and usability.

Dataset Constraints

The dataset used for training lacked:

- Real-world diversity (global IP variations, device fingerprints)
- Long-term behavioral tracking
- High-quality labeled attack data

This limitation reduced the model's ability to generalize across different user profiles and attack scenarios.

Latency in Real-Time Processing

The system introduced minor delays due to:

- Feature extraction overhead
- Machine learning inference time
- Blockchain transaction processing

Although acceptable in a prototype, such latency may become critical in large-scale production environments where thousands of authentication requests occur simultaneously.

Blockchain Overhead

While blockchain ensured:

- Data integrity
- Tamper-proof logging
- Transparency

It also introduced:

- Increased computational cost
- Slower write operations
- Scalability concerns

This trade-off between security and performance must be optimized.

Limited Behavioural Analysis

The current system primarily relies on static contextual features and lacks:

- Continuous behavioural tracking
- Advanced user profiling
- Session-based anomaly detection

This limits the system's ability to detect sophisticated attacks such as session hijacking.

5.3 Future Work (Way Ahead)

To overcome the limitations and further enhance the system, several improvements and research directions are proposed.

Integration of Real-World Datasets

Future implementations should incorporate large-scale, real-world authentication datasets containing:

- Diverse geographic login patterns
- Device fingerprint variations
- Real attack scenarios (e.g., brute force, credential stuffing)

This will significantly improve model accuracy and robustness.

Advanced Machine Learning Models

Instead of traditional ML models, more sophisticated approaches can be used:

- **Deep Learning Models** (e.g., Neural Networks)
- **Recurrent Models (LSTM/GRU)** for time-based behavior analysis
- **Transformer-based Models** for sequence modeling

These models can capture complex user behavior patterns and improve anomaly detection.

Adaptive and Continuous Learning

Implement a **self-learning system** where:

- The model continuously updates based on new login data
- User behavior profiles evolve over time
- Feedback loops improve prediction accuracy

This will enable the system to adapt to changing user patterns dynamically.

Dynamic Multi-Factor Authentication (MFA)

Enhance the system by integrating **risk-based MFA**, where:

- Low-risk logins → Direct access
- Medium-risk → OTP verification
- High-risk → Biometric or multi-step verification

This ensures strong security without affecting user convenience.

Behavioral Biometrics Integration

Incorporate advanced behavioral signals such as:

- Typing speed and keystroke dynamics
- Mouse movement patterns
- Touchscreen gestures

These features provide an additional layer of continuous authentication.

Optimization of Blockchain Layer

To reduce overhead:

- Use **private or permissioned blockchains**
- Implement **off-chain storage with on-chain hashing**
- Optimize transaction batching

This will improve scalability and performance.

Scalability and Cloud Deployment

Deploy the system using:

- **Microservices architecture**
- **Cloud platforms (AWS, Azure, GCP)**
- **Load balancing and distributed processing**

This will enable handling of large-scale user traffic efficiently.

Privacy-Preserving Techniques

To ensure data privacy:

- Implement **Federated Learning** (training without centralizing data)
- Use **Differential Privacy techniques**
- Enable **on-device inference**

This is especially important for sensitive user data.

Integration with IAM Systems

Extend the system to integrate with real-world IAM tools such as:

- LDAP directories
- Identity providers (IdP)
- OAuth/OpenID Connect frameworks

This will make the solution enterprise ready.

5.4 Final Remarks

The proposed AI-based authentication system represents a significant step toward next-generation identity security solutions. By combining machine learning, contextual awareness, and secure logging mechanisms, the system introduces intelligence into authentication workflows that were traditionally static and rule based.

Although certain limitations were observed, they provide valuable insights for future improvements. With advancements in machine learning, data availability, and distributed systems, the proposed framework has the potential to evolve into a highly secure, scalable, and adaptive authentication solution.

In the long term, such intelligent authentication systems can play a crucial role in protecting digital identities, preventing cyber threats, and enabling secure access in increasingly complex and distributed computing environments.

5.5 References

[1] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[2] V. C. Hu, D. Ferraiolo, and D. Kuhn, "Assessment of Access Control Systems," *NIST Interagency Report 7316*, National Institute of Standards and Technology, 2017.

[3] A. Alqahtani, H. Aljohani, and M. Alenazi, "Risk-Based Authentication for Secure Identity Management," *IEEE Access*, vol. 6, pp. 22775–22785, 2018.

[4] M. Zyskind, O. Nathan, and A. Pentland, "Decentralizing Privacy: Using Blockchain to Protect Personal Data," *IEEE Security and Privacy Workshops*, pp. 180–184, 2015.

[5] D. Reed and A. Tobin, "Self-Sovereign Identity," *Decentralized Identity Foundation*, 2016.

[6] Y. Liu, K. Wang, Y. Lin, and W. Xu, "Blockchain-Based Identity Management Systems: A Survey," *Journal of Network and Computer Applications*, vol. 166, 2020.

[7] J. Zhang, X. Chen, and Y. Xiang, "Behavior-Based Access Control Using Machine Learning," *ACM Transactions on Privacy and Security*, vol. 24, no. 2, 2021.

[8] K. Sharma and R. Gupta, "Adaptive Multi-Factor Authentication for Secure Systems," *Computers & Security*, vol. 108, 2022.

[9] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, 2020.

[10] R. Patel, S. Mehta, and A. Verma, "Context-Aware Security Decision Making Using Large Language Models," *IEEE International Conference on Artificial Intelligence and Security*, 2023.

[11] K. Verma, P. Singh, and A. Kumar, "Integrating Artificial Intelligence and Blockchain for Secure Identity Management," *Springer Journal of Information Security*, vol. 12, no. 1, 2024.

[12] Hyperledger Foundation, "Hyperledger Fabric: A Distributed Ledger Platform," *Linux Foundation*, 2023. [Online]. Available: https://www.hyperledger.org

[13] Ethereum Foundation, "Ethereum White Paper," 2014. [Online]. Available: https://ethereum.org

[14] OpenAI, "Large Language Models and Their Applications," *Technical Report*, 2023.

[15] D. Ferraiolo, A. Kuhn, and R. Chandramouli, "Role-Based Access Control," *Artech House*, 2003.

Additional References

[16] X. Jin, R. Krishnan, and R. Sandhu, "A Unified Attribute-Based Access Control Model Covering DAC, MAC and RBAC," *Data and Applications Security and Privacy XXVI*, Springer, 2012.

[17] E. Yuan and J. Tong, "Attribute-Based Access Control (ABAC) for Web Services," *IEEE International Conference on Web Services*, 2005.

[18] NIST, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," NIST SP 800-162, 2014.

[19] J. Kindervag, "Build Security Into Your Network's DNA: The Zero Trust Network Architecture," *Forrester Research*, 2010.

[20] Google, "BeyondCorp: A New Approach to Enterprise Security," Google Whitepaper, 2014.

[21] M. Frank, R. Biedert, E. Ma, I. Martinovic, and D. Song, "Touchalytics: On the Applicability of Touchscreen Input as a Behavioral Biometric," *IEEE Transactions on Information Forensics and Security*, vol. 8, no. 1, 2013.

[22] S. Eberz, K. B. Rasmussen, V. Lenders, and I. Martinovic, "Evaluating Behavioral Biometrics for Continuous Authentication," *ACM Conference on Computer and Communications Security (CCS)*, 2017.

[23] J. Bonneau et al., "The Quest to Replace Passwords: A Framework for Comparative Evaluation," *IEEE Symposium on Security and Privacy*, 2012.

[24] M. Alshehri and F. Coenen, "Predictive Risk-Based Authentication Using Machine Learning," *International Conference on Data Mining*, 2019.

[25] T. Hastie, R. Tibshirani, and J. Friedman, “The Elements of Statistical Learning,” Springer, 2009.

[26] I. Goodfellow, Y. Bengio, and A. Courville, “Deep Learning,” MIT Press, 2016.

[27] A. Ng, “Machine Learning Yearning,” DeepLearning.ai, 2018.

[28] S. Axelsson, “The Base-Rate Fallacy and Its Implications for Intrusion Detection,” *ACM CCS*, 1999.

[29] C. Bishop, “Pattern Recognition and Machine Learning,” Springer, 2006.

[30] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.

[31] G. Wood, “Ethereum: A Secure Decentralized Generalized Transaction Ledger,” 2014.

[32] W3C, “Decentralized Identifiers (DIDs) v1.0,” World Wide Web Consortium, 2022.

[33] W3C, “Verifiable Credentials Data Model 1.0,” 2019.

[34] A. Tobin and D. Reed, “The Inevitable Rise of Self-Sovereign Identity,” The Sovrin Foundation, 2017.

[35] H. B. McMahan et al., “Communication-Efficient Learning of Deep Networks from Decentralized Data,” *AISTATS*, 2017.

[36] C. Dwork, “Differential Privacy,” *International Colloquium on Automata, Languages and Programming (ICALP)*, 2006.

[37] P. Kairouz et al., “Advances and Open Problems in Federated Learning,” *Foundations and Trends in Machine Learning*, 2021.

[38] N. Carlini et al., “Extracting Training Data from Large Language Models,” *USENIX Security Symposium*, 2021.

[39] OpenAI, “GPT-4 Technical Report,” 2023.

[40] Y. Liu et al., “Trustworthy AI: A Survey on Security and Privacy,” *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[41] Red Hat, “Keycloak Documentation,” 2023.

[42] OpenLDAP Foundation, “OpenLDAP Software 2.6 Administrator’s Guide,” 2022.

[43] Microsoft, “Azure Active Directory Security Architecture,” 2023.

[44] Amazon Web Services, “AWS Identity and Access Management (IAM) Best Practices,” 2023.